

Human review packet: LMMS04FairDivision

Generated from byte-pinned source excerpts, the expanded PaperInterface specifications, and hash-bound raw-source-to-Spec screening records.

Paper: LMMS04FairDivision
Paper id: LMMS04FairDivision
Source version: not recorded
Source record: audit/paper_statement_map.json
Rows: 31
Generated: 2026-08-18
Source URL: [official source](#)

Review status: 0/31 source-claim screens; 0/60 library prerequisite screens. This is a review worksheet while those direct checks remain pending; it is not a final validation receipt.

How to use this packet

For each source claim, compare the exact source excerpts with the expanded PaperInterface specification. Mark up this PDF or fill the blank reviewer box in the TeX source; return the annotated file for reconciliation into the paper-local human-review log.

Open the interactive dashboard

The interactive dashboard is an optional alternative to using this PDF; it presents the same review surface rather than an additional review requirement. After forking and cloning (or pulling) EconCSLib, run the following from the repository root. The cache command is needed only the first time in a new clone.

```
cd <your-EconCSLib-checkout>
lake exe cache get
python3 scripts/review_dashboard.py --paper LMMS04FairDivision --serve
```

Open <http://127.0.0.1:8765/> in a browser and keep that terminal running while reviewing. The dashboard records saved annotations in this paper's reviewer-owned review record.

Contents

- [Semantic library prerequisites \(60; not paper claims\)](#)
 - [EconCSLib.IsMeasureAtom](#)
 - [EconCSLib.AtomsBoundedBy](#)
 - [Bundle Item](#)
 - [Allocation Agent Item](#)
 - [EconCSLib.FairDivision.Valuation](#)
 - [EconCSLib.FairDivision.Valuation.value](#)
 - [EconCSLib.FairDivision.EnvyEdge](#)
 - [EconCSLib.FairDivision.AcyclicEnvyGraph](#)
 - [EconCSLib.FairDivision.DirectFairDivisionMechanism](#)
 - [EconCSLib.FairDivision.FairDivisionReport](#)
 - [EconCSLib.FairDivision.DirectFairDivisionMechanism.allocation](#)
 - [EconCSLib.FairDivision.DirectFairDivisionMechanism.utility](#)
 - [EconCSLib.FairDivision.DirectFairDivisionMechanism.Truthful](#)
 - [EconCSLib.FairDivision.envy](#)
 - [EconCSLib.FairDivision.EnvyBoundedBy](#)
 - [IsAllocationOf allocation chores](#)
 - [EconCSLib.FairDivision.NoOneEnvies](#)
 - [EconCSLib.FairDivision.HasUnenviedReduction](#)
 - [EconCSLib.FairDivision.MarginalBound](#)
 - [EconCSLib.FairDivision.MeasurePartitionCertificate](#)
 - [EconCSLib.FairDivision.MeasurePartitionCertificate.mk](#)
 - [EconCSLib.FairDivision.MeasurePartitionCertificate.pieceSet](#)
 - [EconCSLib.FairDivision.MeasurePartitionCertificate.weight](#)
 - [EconCSLib.FairDivision.MeasurePartitionCertificate.weight_nonneg](#)
 - [EconCSLib.FairDivision.Valuation.mk](#)
 - [EconCSLib.FairDivision.additiveValue](#)
 - [EconCSLib.FairDivision.additiveValuation](#)

- EconCSLib.FairDivision.MeasurePartitionCertificate.valuation
- EconCSLib.FairDivision.reportEnvy
- EconCSLib.FairDivision.maxReportEnvy
- EconCSLib.FairDivision.MinimumReportEnvyAllocation
- EconCSLib.FairDivision.RandomizedDirectFairDivisionMechanism
- EconCSLib.FairDivision.RandomizedDirectFairDivisionMechanism.allocationLaw
- EconCSLib.pmfExp
- EconCSLib.FairDivision.RandomizedDirectFairDivisionMechanism.expectedUtility
- EconCSLib.FairDivision.RandomizedDirectFairDivisionMechanism.Truthful
- EconCSLib.FairDivision.RandomizedDirectFairDivisionMechanism.mk
- EconCSLib.FairDivision.ReportEnvyFree
- EconCSLib.FairDivision.ReturnsAllocationOf
- EconCSLib.FairDivision.ReturnsEnvyFreeWheneverExists
- EconCSLib.FairDivision.ReturnsMinimumReportEnvy
- EconCSLib.FairDivision.addItem
- EconCSLib.FairDivision.aggregateMeasure
- EconCSLib.FairDivision.envyBoundedBy_addItem_of_unenvied
- EconCSLib.FairDivision.isAllocationOf_addItem_insert
- EconCSLib.FairDivision.boundedEnvyStep
- EconCSLib.FairDivision.emptyAllocation
- EconCSLib.FairDivision.boundedEnvyAlgorithm
- EconCSLib.FairDivision.finiteSingletonsWithResidualPieceSet
- EconCSLib.FairDivision.maxMarginal
- EconCSLib.FairDivision.measurePartitionCertificateOfFiniteSingletonsAndResidualAggregate
- EconCSLib.Math.TendsToZero
- EconCSLib.Probability.RealIntervalPartition
- EconCSLib.Probability.RealIntervalPartition.Piece
- EconCSLib.Probability.RealIntervalPartition.instDecidableEq
- EconCSLib.Probability.RealIntervalPartition.instFintype
- EconCSLib.Probability.RealIntervalPartition.pieceSet
- EconCSLib.pmfProb
- EconCSLib.pmfProduct
- EconCSLib.uniformPMF
- Source claims (31)
 - envySpec
 - envyFreeSpec
 - envyBoundedBySpec
 - maxMarginalSpec
 - isAllocationOfSpec
 - lemma2_2_acyclic_reductionSpec
 - theorem2_1_bounded_envy_allocation_existsSpec
 - theorem2_1_alpha_bounded_allocation_existsSpec
 - theorem2_1_algorithm_correct_list_toFinsetSpec
 - theorem2_3_real_interval_supported_atom_boundSpec
 - theorem3_1_eventually_minimum_envy_lower_bound_from_twoBit_adaptive_queriesSpec
 - theorem3_1_eventually_minimum_envy_ratio_lower_bound_from_twoBit_adaptive_queriesSpec
 - theorem3_2_graham_certificate_to_envy_ratio_boundSpec
 - theorem3_2_graham_factor_eq_seven_fifthsSpec
 - theorem3_3_solver_auto_cap_full_ip_summary_with_ratio_guaranteeSpec
 - theorem3_3_claim34_fixed_rounding_ratio_endpointSpec
 - theorem3_3_claim34_capped_weighted_supply_ratio_endpointSpec
 - claim3_4_bounded_optimal_of_exact_allocations_with_nonempty_positive_goodsSpec
 - claim3_4_identical_utilities_bounded_optimum_boundSpec
 - theorem3_3_ratio_transfer_certificate_epsilon_of_agentwise_additive_loadsSpec
 - lemma3_5_additive_transfer_epsilon_boundSpec
 - directMechanism_fieldsSpec

- randomizedDirectMechanism_fieldsSpec
- truthfulSpec
- randomizedTruthfulSpec
- theorem4_1_source_goods_contentSpec
- theorem4_1_true_report_formulaSpec
- theorem4_1_source_not_truthful_envy_free_whenenever_existsSpec
- theorem4_1_source_minimum_envy_not_truthfulSpec
- theorem4_2_uniform_random_mechanism_truthfulSpec
- theorem4_2_uniform_random_max_envy_probability_boundSpec

Material library prerequisites

EconCSLib.IsMeasureAtom

Source connection: no explicit byte-pinned paper source connection is registered.

Lean-expanded library semantic target

$\forall \{\Omega : \text{Type } u_1\} [\text{inst} : \text{MeasurableSpace } \Omega] (\mu : \text{MeasureTheory.Measure } \Omega) (S : \text{Set } \Omega),$
 $\mu S \neq 0 \wedge \forall E \subset S, \mu E = 0 \vee \mu E = \mu S$

Exact Lean library declaration

`def IsMeasureAtom (μ : Measure Ω) (S : Set Ω) : Prop :=`
`μ S ≠ 0 ∧ ∀ E : Set Ω, E ⊂ S → μ E = 0 ∨ μ E = μ S`

Recorded source-to-library screening Verdict: not recorded

Reviewer check: ☐ Matches source input

If left unchecked, explain the discrepancy or uncertainty below.

Reviewer annotation

EconCSLib.AtomsBoundedBy

Source connection: no explicit byte-pinned paper source connection is registered.

Lean-expanded library semantic target

```
∀ {Ω : Type u_1} [inst : MeasurableSpace Ω] (μ : MeasureTheory.Measure Ω) (α : ℝ) (S : Set Ω),  
  EconCSLib.IsMeasureAtom μ S → μ.real S ≤ α
```

Exact Lean library declaration

```
def AtomsBoundedBy (μ : Measure Ω) (α : ℝ) : Prop :=  
  ∀ S : Set Ω, IsMeasureAtom μ S → μ.real S ≤ α
```

Recorded source-to-library screening Verdict: not recorded

Reviewer check: ☐ Matches source input

If left unchecked, explain the discrepancy or uncertainty below.

Reviewer annotation

Bundle Item

Source connection: no explicit byte-pinned paper source connection is registered.

Lean-expanded library semantic target

(Item : Type u_1) → Finset Item

Exact Lean library declaration

```
/-- A bundle of indivisible goods. -/  
abbrev Bundle (Item : Type*) := Finset Item
```

Recorded source-to-library screening Verdict: not recorded

Reviewer check: ☐ Matches source input

If left unchecked, explain the discrepancy or uncertainty below.

Reviewer annotation

Allocation Agent Item

Source connection: no explicit byte-pinned paper source connection is registered.

Lean-expanded library semantic target

(Agent : Type u_1) → (Item : Type u_2) → Agent → EconCSLib.FairDivision.Bundle Item

Exact Lean library declaration

```
/-- An allocation assigns each agent a bundle. -/  
abbrev Allocation (Agent Item : Type*) := Agent → Bundle Item
```

Recorded source-to-library screening Verdict: not recorded

Reviewer check: ☐ Matches source input

If left unchecked, explain the discrepancy or uncertainty below.

Reviewer annotation

EconCSLib.FairDivision.Valuation

Source connection: no explicit byte-pinned paper source connection is registered.

Lean declaration type (metadata)

Type u_1 → Type u_2 → Type (max u_1 u_2)

Exact Lean library declaration

```
structure Valuation (Agent Item : Type*) where
  value : Agent → Bundle Item → ℝ
  monotone : ∀ agent {S T : Bundle Item}, S ⊆ T → value agent S ≤ value agent T
```

Recorded source-to-library screening Verdict: not recorded

Reviewer check: ☐ Matches source input

If left unchecked, explain the discrepancy or uncertainty below.

Reviewer annotation

EconCSLib.FairDivision.Valuation.value

Source connection: no explicit byte-pinned paper source connection is registered.

Lean-expanded library semantic target

```
{Agent : Type u_1} →  
  {Item : Type u_2} →  
    (self : EconCSLib.FairDivision.Valuation Agent Item) →  
      (a : Agent) → (a_1 : EconCSLib.FairDivision.Bundle Item) → self.1 a a_1
```

Exact Lean library declaration

library declaration is not registered or discoverable

Recorded source-to-library screening Verdict: not recorded

Reviewer check: ☐ Matches source input

If left unchecked, explain the discrepancy or uncertainty below.

Reviewer annotation

EconCSLib.FairDivision.EnvyEdge

Source connection: no explicit byte-pinned paper source connection is registered.

Lean-expanded library semantic target

```
∀ {Agent : Type u_1} {Item : Type u_2} (v : EconCSLib.FairDivision.Valuation Agent Item)
  (A : EconCSLib.FairDivision.Allocation Agent Item) (i j : Agent), v.value i (A i) < v.value i (A j)
```

Exact Lean library declaration

```
def EnvyEdge (v : Valuation Agent Item) (A : Allocation Agent Item)
  (i j : Agent) : Prop :=
  v.value i (A i) < v.value i (A j)
```

Recorded source-to-library screening Verdict: not recorded

Reviewer check: ☐ Matches source input

If left unchecked, explain the discrepancy or uncertainty below.

Reviewer annotation

EconCSLib.FairDivision.AcyclicEnvyGraph

Source connection: no explicit byte-pinned paper source connection is registered.

Lean-expanded library semantic target

```
∀ {Agent : Type u_1} {Item : Type u_2} (v : EconCSLib.FairDivision.Valuation Agent Item)
  (A : EconCSLib.FairDivision.Allocation Agent Item), WellFounded fun i j => EconCSLib.FairDivision.EnvyEdge v A i j
```

Exact Lean library declaration

```
def AcyclicEnvyGraph (v : Valuation Agent Item) (A : Allocation Agent Item) :
  Prop :=
  WellFounded fun i j => EnvyEdge v A i j
```

Recorded source-to-library screening Verdict: not recorded

Reviewer check: ☐ Matches source input

If left unchecked, explain the discrepancy or uncertainty below.

Reviewer annotation

EconCSLib.FairDivision.DirectFairDivisionMechanism

Source connection: no explicit byte-pinned paper source connection is registered.

Lean declaration type (metadata)

Type u_3 → Type u_4 → Type (max u_3 u_4)

Exact Lean library declaration

```
structure DirectFairDivisionMechanism (Agent Item : Type*) where
  allocation : FairDivisionReport Agent Item → Allocation Agent Item
```

Recorded source-to-library screening Verdict: not recorded

Reviewer check: ☐ Matches source input

If left unchecked, explain the discrepancy or uncertainty below.

Reviewer annotation

EconCSLib.FairDivision.FairDivisionReport

Source connection: no explicit byte-pinned paper source connection is registered.

Lean-expanded library semantic target

(Agent : Type u_3) → (Item : Type u_4) → Agent → EconCSLib.FairDivision.Bundle Item → $\mathbb{R}$

Exact Lean library declaration

```
abbrev FairDivisionReport (Agent Item : Type*) :=  
  Agent → Bundle Item →  $\mathbb{R}$ 
```

Recorded source-to-library screening Verdict: not recorded

Reviewer check: ☐ Matches source input

If left unchecked, explain the discrepancy or uncertainty below.

Reviewer annotation

EconCSLib.FairDivision.DirectFairDivisionMechanism.allocation

Source connection: no explicit byte-pinned paper source connection is registered.

Lean-expanded library semantic target

```
{Agent : Type u_3} →  
  {Item : Type u_4} →  
    (self : EconCSLib.FairDivision.DirectFairDivisionMechanism Agent Item) →  
      (a : EconCSLib.FairDivision.FairDivisionReport Agent Item) → (a_1 : Agent) → self.1 a a_1
```

Exact Lean library declaration

library declaration is not registered or discoverable

Recorded source-to-library screening Verdict: not recorded

Reviewer check: ☐ Matches source input

If left unchecked, explain the discrepancy or uncertainty below.

Reviewer annotation

EconCSLib.FairDivision.DirectFairDivisionMechanism.utility

Source connection: no explicit byte-pinned paper source connection is registered.

Lean-expanded library semantic target

```
{Agent : Type u_1} →  
  {Item : Type u_2} →  
    (M : EconCSLib.FairDivision.DirectFairDivisionMechanism Agent Item) →  
      (values reports : EconCSLib.FairDivision.FairDivisionReport Agent Item) →  
        (i : Agent) → values i (M.allocation reports i)
```

Exact Lean library declaration

```
def utility (M : DirectFairDivisionMechanism Agent Item)  
  (values reports : FairDivisionReport Agent Item) (i : Agent) : ℝ :=  
  values i (M.allocation reports i)
```

Recorded source-to-library screening Verdict: not recorded

Reviewer check: ☐ Matches source input

If left unchecked, explain the discrepancy or uncertainty below.

Reviewer annotation

EconCSLib.FairDivision.DirectFairDivisionMechanism.Truthful

Source connection: no explicit byte-pinned paper source connection is registered.

Lean-expanded library semantic target

```
∀ {Agent : Type u_1} {Item : Type u_2} [inst : DecidableEq Agent]
  (M : EconCSLib.FairDivision.DirectFairDivisionMechanism Agent Item)
  (values : EconCSLib.FairDivision.FairDivisionReport Agent Item) (i : Agent)
  (report : EconCSLib.FairDivision.Bundle Item → ℝ),
  M.utility values (Function.update values i report) i ≤ M.utility values values i
```

Exact Lean library declaration

```
def Truthful [DecidableEq Agent]
  (M : DirectFairDivisionMechanism Agent Item) : Prop :=
  ∀ (values : FairDivisionReport Agent Item)
    (i : Agent) (report : Bundle Item → ℝ),
    M.utility values (Function.update values i report) i ≤
    M.utility values values i
```

Recorded source-to-library screening Verdict: not recorded

Reviewer check: ☐ Matches source input

If left unchecked, explain the discrepancy or uncertainty below.

Reviewer annotation

EconCSLib.FairDivision.envy

Source connection: no explicit byte-pinned paper source connection is registered.

Lean-expanded library semantic target

```
{Agent : Type u_1} →  
{Item : Type u_2} →  
  (v : EconCSLib.FairDivision.Valuation Agent Item) →  
  (A : EconCSLib.FairDivision.Allocation Agent Item) → (i j : Agent) → max 0 (v.value i (A j) - v.value i (A i))
```

Exact Lean library declaration

```
def envy (v : Valuation Agent Item) (A : Allocation Agent Item) (i j : Agent) : ℝ :=  
  max 0 (v.value i (A j) - v.value i (A i))
```

Recorded source-to-library screening Verdict: not recorded

Reviewer check: ☐ Matches source input

If left unchecked, explain the discrepancy or uncertainty below.

Reviewer annotation

EconCSLib.FairDivision.EnvyBoundedBy

Source connection: no explicit byte-pinned paper source connection is registered.

Lean-expanded library semantic target

```
∀ {Agent : Type u_1} {Item : Type u_2} (v : EconCSLib.FairDivision.Valuation Agent Item)
  (A : EconCSLib.FairDivision.Allocation Agent Item) (α : ℝ) (i j : Agent), EconCSLib.FairDivision.envy v A i j ≤ α
```

Exact Lean library declaration

```
def EnvyBoundedBy (v : Valuation Agent Item) (A : Allocation Agent Item) (α : ℝ) : Prop :=
  ∀ i j, envy v A i j ≤ α
```

Recorded source-to-library screening Verdict: not recorded

Reviewer check: ☐ Matches source input

If left unchecked, explain the discrepancy or uncertainty below.

Reviewer annotation

IsAllocationOf allocation chores

Source connection: no explicit byte-pinned paper source connection is registered.

Lean-expanded library semantic target

```
∀ {Agent : Type u_1} {Item : Type u_2} [DecidableEq Item] (A : EconCSLib.FairDivision.Allocation Agent Item)
  (goods : Finset Item), (∀ (i : Agent), ∀ g ∈ A i, g ∈ goods) ∧ ∀ g ∈ goods, ∃! i, g ∈ A i
```

Exact Lean library declaration

```
/-- `A` allocates exactly the goods in `goods`, with no duplicates and no outsiders. -/
def IsAllocationOf [DecidableEq Item] (A : Allocation Agent Item) (goods : Finset Item) : Prop :=
  (∀ i g, g ∈ A i → g ∈ goods) ∧
  ∀ g, g ∈ goods → ∃! i, g ∈ A i
```

Recorded source-to-library screening Verdict: not recorded

Reviewer check: ☐ Matches source input

If left unchecked, explain the discrepancy or uncertainty below.

Reviewer annotation

EconCSLib.FairDivision.NoOneEnvies

Source connection: no explicit byte-pinned paper source connection is registered.

Lean-expanded library semantic target

```
∀ {Agent : Type u_1} {Item : Type u_2} (v : EconCSLib.FairDivision.Valuation Agent Item)
  (A : EconCSLib.FairDivision.Allocation Agent Item) (owner i : Agent), v.value i (A owner) ≤ v.value i (A i)
```

Exact Lean library declaration

```
def NoOneEnvies (v : Valuation Agent Item) (A : Allocation Agent Item)
  (owner : Agent) : Prop :=
  ∀ i, v.value i (A owner) ≤ v.value i (A i)
```

Recorded source-to-library screening Verdict: not recorded

Reviewer check: ☐ Matches source input

If left unchecked, explain the discrepancy or uncertainty below.

Reviewer annotation

EconCSLib.FairDivision.HasUnenviedReduction

Source connection: no explicit byte-pinned paper source connection is registered.

Lean-expanded library semantic target

```
∀ {Agent : Type u_1} {Item : Type u_2} [inst : DecidableEq Item] (v : EconCSLib.FairDivision.Valuation Agent Item)
  (α : ℝ) (goods : Finset Item) (A : EconCSLib.FairDivision.Allocation Agent Item),
  EconCSLib.FairDivision.IsAllocationOf A goods →
    EconCSLib.FairDivision.EnvyBoundedBy v A α →
      ∃ B owner,
        EconCSLib.FairDivision.IsAllocationOf B goods ∧
          EconCSLib.FairDivision.EnvyBoundedBy v B α ∧ EconCSLib.FairDivision.NoOneEnvies v B owner
```

Exact Lean library declaration

```
def HasUnenviedReduction [DecidableEq Item]
  (v : Valuation Agent Item) (α : ℝ) : Prop :=
  ∀ goods (A : Allocation Agent Item),
    IsAllocationOf A goods →
      EnvyBoundedBy v A α →
        ∃ B owner, IsAllocationOf B goods ∧ EnvyBoundedBy v B α ∧ NoOneEnvies v B owner
```

Recorded source-to-library screening Verdict: not recorded

Reviewer check: ☐ Matches source input

If left unchecked, explain the discrepancy or uncertainty below.

Reviewer annotation

EconCSLib.FairDivision.MarginalBound

Source connection: no explicit byte-pinned paper source connection is registered.

Lean-expanded library semantic target

```
∀ {Agent : Type u_1} {Item : Type u_2} [inst : DecidableEq Item] (v : EconCSLib.FairDivision.Valuation Agent Item)
  (α : ℝ) (agent : Agent) (S : EconCSLib.FairDivision.Bundle Item) (g : Item),
  v.value agent (insert g S) - v.value agent S ≤ α
```

Exact Lean library declaration

```
def MarginalBound [DecidableEq Item] (v : Valuation Agent Item) (α : ℝ) : Prop :=
  ∀ agent (S : Bundle Item) (g : Item),
    v.value agent (insert g S) - v.value agent S ≤ α
```

Recorded source-to-library screening Verdict: not recorded

Reviewer check: ☐ Matches source input

If left unchecked, explain the discrepancy or uncertainty below.

Reviewer annotation

EconCSLib.FairDivision.MeasurePartitionCertificate

Source connection: no explicit byte-pinned paper source connection is registered.

Lean declaration type (metadata)

```
{Agent : Type u_1} →  
{Ω : Type u_2} → [inst : MeasurableSpace Ω] → (Agent → MeasureTheory.Measure Ω) → ℝ → Type u_5 → Type (max u_2 u_5)
```

Exact Lean library declaration

```
structure MeasurePartitionCertificate  
  (μ : Agent → Measure Ω) (α : ℝ) (Piece : Type*) where  
  pieceSet : Piece → Set Ω  
  measurable_piece : ∀ piece, MeasurableSet (pieceSet piece)  
  pairwiseDisjoint : Set.PairwiseDisjoint Set.univ pieceSet  
  cover : Set.iUnion pieceSet = Set.univ  
  piece_value_le : ∀ agent piece, (μ agent).real (pieceSet piece) ≤ α
```

Recorded source-to-library screening Verdict: not recorded

Reviewer check: ☐ Matches source input

If left unchecked, explain the discrepancy or uncertainty below.

Reviewer annotation

EconCSLib.FairDivision.MeasurePartitionCertificate.mk

Source connection: no explicit byte-pinned paper source connection is registered.

Lean declaration type (metadata)

```
{Agent : Type u_1} →
{Ω : Type u_2} →
[inst : MeasurableSpace Ω] →
{μ : Agent → MeasureTheory.Measure Ω} →
{α : ℝ} →
{Piece : Type u_5} →
(pieceSet : Piece → Set Ω) →
(∀ (piece : Piece), MeasurableSet (pieceSet piece)) →
Set.univ.PairwiseDisjoint pieceSet →
Set.iUnion pieceSet = Set.univ →
(∀ (agent : Agent) (piece : Piece), (μ agent).real (pieceSet piece) ≤ α) →
EconCSLib.FairDivision.MeasurePartitionCertificate μ α Piece
```

Exact Lean library declaration

library declaration is not registered or discoverable

Recorded source-to-library screening Verdict: not recorded

Reviewer check: ☐ Matches source input

If left unchecked, explain the discrepancy or uncertainty below.

Reviewer annotation

EconCSLib.FairDivision.MeasurePartitionCertificate.pieceSet

Source connection: no explicit byte-pinned paper source connection is registered.

Lean-expanded library semantic target

```
∀ {Agent : Type u_1} {Ω : Type u_2} [inst : MeasurableSpace Ω] {μ : Agent → MeasureTheory.Measure Ω} {α : ℝ}
  {Piece : Type u_5} (self : EconCSLib.FairDivision.MeasurePartitionCertificate μ α Piece) (a : Piece) (a_1 : Ω),
  self.1 a a_1
```

Exact Lean library declaration

library declaration is not registered or discoverable

Recorded source-to-library screening Verdict: not recorded

Reviewer check: ☐ Matches source input

If left unchecked, explain the discrepancy or uncertainty below.

Reviewer annotation

EconCSLib.FairDivision.MeasurePartitionCertificate.weight

Source connection: no explicit byte-pinned paper source connection is registered.

Lean-expanded library semantic target

```
{Agent : Type u_1} →
{Ω : Type u_2} →
{Piece : Type u_3} →
[inst : MeasurableSpace Ω] →
{μ : Agent → MeasureTheory.Measure Ω} →
{α : ℝ} →
(C : EconCSLib.FairDivision.MeasurePartitionCertificate μ α Piece) →
(agent : Agent) → (piece : Piece) → (μ agent).real (C.pieceSet piece)
```

Exact Lean library declaration

```
noncomputable def weight (C : MeasurePartitionCertificate μ α Piece)
  (agent : Agent) (piece : Piece) : ℝ :=
  (μ agent).real (C.pieceSet piece)
```

Recorded source-to-library screening Verdict: not recorded

Reviewer check: ☐ Matches source input

If left unchecked, explain the discrepancy or uncertainty below.

Reviewer annotation

EconCSLib.FairDivision.MeasurePartitionCertificate.weight_nonneg

Source connection: no explicit byte-pinned paper source connection is registered.

Lean declaration type (metadata)

```
∀ {Agent : Type u_1} {Ω : Type u_2} {Piece : Type u_3} [inst : MeasurableSpace Ω] {μ : Agent → MeasureTheory.Measure Ω}
  {α : ℝ} (C : EconCSLib.FairDivision.MeasurePartitionCertificate μ α Piece) (agent : Agent) (piece : Piece),
  0 ≤ C.weight agent piece
```

Exact Lean library declaration

```
theorem weight_nonneg (C : MeasurePartitionCertificate μ α Piece)
  (agent : Agent) (piece : Piece) :
  0 ≤ C.weight agent piece
```

Recorded source-to-library screening Verdict: not recorded

Reviewer check: ☐ Matches source input

If left unchecked, explain the discrepancy or uncertainty below.

Reviewer annotation

EconCSLib.FairDivision.Valuation.mk

Source connection: no explicit byte-pinned paper source connection is registered.

Lean declaration type (metadata)

```
{Agent : Type u_1} →  
  {Item : Type u_2} →  
    (value : Agent → EconCSLib.FairDivision.Bundle Item → ℝ) →  
      (∀ (agent : Agent) {S T : EconCSLib.FairDivision.Bundle Item}, S ≤ T → value agent S ≤ value agent T) →  
        EconCSLib.FairDivision.Valuation Agent Item
```

Exact Lean library declaration

library declaration is not registered or discoverable

Recorded source-to-library screening Verdict: not recorded

Reviewer check: ☐ Matches source input

If left unchecked, explain the discrepancy or uncertainty below.

Reviewer annotation

EconCSLib.FairDivision.additiveValue

Source connection: no explicit byte-pinned paper source connection is registered.

Lean-expanded library semantic target

```
{Agent : Type u_1} →  
{Item : Type u_2} →  
  (w : Agent → Item → ℝ) → (agent : Agent) → (S : EconCSLib.FairDivision.Bundle Item) →  $\sum g \in S, w \text{ agent } g$ 
```

Exact Lean library declaration

```
noncomputable def additiveValue (w : Agent → Item → ℝ)  
  (agent : Agent) (S : Bundle Item) : ℝ :=  
  S.sum fun g => w agent g
```

Recorded source-to-library screening Verdict: not recorded

Reviewer check: ☐ Matches source input

If left unchecked, explain the discrepancy or uncertainty below.

Reviewer annotation

EconCSLib.FairDivision.additiveValuation

Source connection: no explicit byte-pinned paper source connection is registered.

Lean-expanded library semantic target

```
{Agent : Type u_1} →
{Item : Type u_2} →
[DecidableEq Item] →
(w : Agent → Item → ℝ) →
(hnonneg : ∀ (agent : Agent) (g : Item), 0 ≤ w agent g) →
{ value := EconCSLib.FairDivision.additiveValue w, monotone := ... }
```

Exact Lean library declaration

```
noncomputable def additiveValuation [DecidableEq Item]
(w : Agent → Item → ℝ) (hnonneg : ∀ agent g, 0 ≤ w agent g) :
Valuation Agent Item where
value := additiveValue w
monotone := by
intro agent S T hsub
unfold additiveValue
exact Finset.sum_le_sum_of_subset_of_nonneg hsub
(by intro g _ _; exact hnonneg agent g)
```

Recorded source-to-library screening Verdict: not recorded

Reviewer check: ☐ Matches source input

If left unchecked, explain the discrepancy or uncertainty below.

Reviewer annotation

EconCSLib.FairDivision.MeasurePartitionCertificate.valuation

Source connection: no explicit byte-pinned paper source connection is registered.

Lean-expanded library semantic target

```
{Agent : Type u_1} →
{Ω : Type u_2} →
{Piece : Type u_3} →
[inst : MeasurableSpace Ω] →
{μ : Agent → MeasureTheory.Measure Ω} →
{α : ℝ} →
[inst_1 : DecidableEq Piece] →
(C : EconCSLib.FairDivision.MeasurePartitionCertificate μ α Piece) →
EconCSLib.FairDivision.additiveValuation C.weight ...
```

Exact Lean library declaration

```
noncomputable def valuation [DecidableEq Piece]
  (C : MeasurePartitionCertificate μ α Piece) :
  Valuation Agent Piece :=
  additiveValuation C.weight C.weight_nonneg
```

Recorded source-to-library screening Verdict: not recorded

Reviewer check: ☐ Matches source input

If left unchecked, explain the discrepancy or uncertainty below.

Reviewer annotation

EconCSLib.FairDivision.reportEnvy

Source connection: no explicit byte-pinned paper source connection is registered.

Lean-expanded library semantic target

```
{Agent : Type u_1} →  
  {Item : Type u_2} →  
    (values : EconCSLib.FairDivision.FairDivisionReport Agent Item) →  
      (A : EconCSLib.FairDivision.Allocation Agent Item) → (i j : Agent) → max 0 (values i (A j) - values i (A i))
```

Exact Lean library declaration

```
def reportEnvy (values : FairDivisionReport Agent Item)  
  (A : Allocation Agent Item) (i j : Agent) : ℝ :=  
  max 0 (values i (A j) - values i (A i))
```

Recorded source-to-library screening Verdict: not recorded

Reviewer check: ☐ Matches source input

If left unchecked, explain the discrepancy or uncertainty below.

Reviewer annotation

EconCSLib.FairDivision.maxReportEnvy

Source connection: no explicit byte-pinned paper source connection is registered.

Lean-expanded library semantic target

```
{Agent : Type u_1} →
{Item : Type u_2} →
[inst : Fintype Agent] →
[inst_1 : Nonempty Agent] →
(values : EconCSLib.FairDivision.FairDivisionReport Agent Item) →
(A : EconCSLib.FairDivision.Allocation Agent Item) →
(Finset.univ.product Finset.univ).sup' ~ fun p => EconCSLib.FairDivision.reportEnvy values A p.1 p.2
```

Exact Lean library declaration

```
noncomputable def maxReportEnvy [Fintype Agent] [Nonempty Agent]
  (values : FairDivisionReport Agent Item) (A : Allocation Agent Item) : ℝ :=
  ((Finset.univ : Finset Agent).product (Finset.univ : Finset Agent)).sup'
  (by
    obtain {i} := (inferInstance : Nonempty Agent)
    exact {(i, i), by simp})
  (fun p => reportEnvy values A p.1 p.2)
```

Recorded source-to-library screening Verdict: not recorded

Reviewer check: ☐ Matches source input

If left unchecked, explain the discrepancy or uncertainty below.

Reviewer annotation

EconCSLib.FairDivision.MinimumReportEnvyAllocation

Source connection: no explicit byte-pinned paper source connection is registered.

Lean-expanded library semantic target

```
∀ {Agent : Type u_1} {Item : Type u_2} [inst : Fintype Agent] [inst_1 : Nonempty Agent] [inst_2 : DecidableEq Item]
  (values : EconCSLib.FairDivision.FairDivisionReport Agent Item) (goods : Finset Item)
  (A : EconCSLib.FairDivision.Allocation Agent Item),
  EconCSLib.FairDivision.IsAllocationOf A goods ∧
  ∀ (B : EconCSLib.FairDivision.Allocation Agent Item),
    EconCSLib.FairDivision.IsAllocationOf B goods →
      EconCSLib.FairDivision.maxReportEnvy values A ≤ EconCSLib.FairDivision.maxReportEnvy values B
```

Exact Lean library declaration

```
def MinimumReportEnvyAllocation [Fintype Agent] [Nonempty Agent] [DecidableEq Item]
  (values : FairDivisionReport Agent Item) (goods : Finset Item)
  (A : Allocation Agent Item) : Prop :=
  IsAllocationOf A goods ∧
  ∀ B : Allocation Agent Item,
    IsAllocationOf B goods →
      maxReportEnvy values A ≤ maxReportEnvy values B
```

Recorded source-to-library screening Verdict: not recorded

Reviewer check: ☐ Matches source input

If left unchecked, explain the discrepancy or uncertainty below.

Reviewer annotation

EconCSLib.FairDivision.RandomizedDirectFairDivisionMechanism

Source connection: no explicit byte-pinned paper source connection is registered.

Lean declaration type (metadata)

Type u_3 → Type u_4 → Type (max u_3 u_4)

Exact Lean library declaration

```
structure RandomizedDirectFairDivisionMechanism (Agent Item : Type*) where
  allocationLaw :
    FairDivisionReport Agent Item → PMF (Allocation Agent Item)
```

Recorded source-to-library screening Verdict: not recorded

Reviewer check: ☐ Matches source input

If left unchecked, explain the discrepancy or uncertainty below.

Reviewer annotation

EconCSLib.FairDivision.RandomizedDirectFairDivisionMechanism.allocationLaw

Source connection: no explicit byte-pinned paper source connection is registered.

Lean-expanded library semantic target

```
{Agent : Type u_3} →  
  {Item : Type u_4} →  
    (self : EconCSLib.FairDivision.RandomizedDirectFairDivisionMechanism Agent Item) →  
      (a : EconCSLib.FairDivision.FairDivisionReport Agent Item) → self.1 a
```

Exact Lean library declaration

library declaration is not registered or discoverable

Recorded source-to-library screening Verdict: not recorded

Reviewer check: ☐ Matches source input

If left unchecked, explain the discrepancy or uncertainty below.

Reviewer annotation

EconCSLib.pmfExp

Source connection: no explicit byte-pinned paper source connection is registered.

Lean-expanded library semantic target

$\{\alpha : \text{Type } u_1\} \rightarrow [\text{inst} : \text{Fintype } \alpha] \rightarrow [\text{DecidableEq } \alpha] \rightarrow (\mu : \text{PMF } \alpha) \rightarrow (f : \alpha \rightarrow \mathbb{R}) \rightarrow \sum a, (\mu a).toReal * f a$

Exact Lean library declaration

```
noncomputable def pmfExp {α : Type*} [Fintype α] [DecidableEq α]
  (μ : PMF α) (f : α → ℝ) : ℝ :=
  ∑ a, (μ a).toReal * f a
```

Recorded source-to-library screening Verdict: not recorded

Reviewer check: ☐ Matches source input

If left unchecked, explain the discrepancy or uncertainty below.

Reviewer annotation

EconCSLib.FairDivision.RandomizedDirectFairDivisionMechanism.expectedUtility

Source connection: no explicit byte-pinned paper source connection is registered.

Lean-expanded library semantic target

```
{Agent : Type u_1} →
{Item : Type u_2} →
  [inst : Fintype (EconCSLib.FairDivision.Allocation Agent Item)] →
  [inst_1 : DecidableEq (EconCSLib.FairDivision.Allocation Agent Item)] →
  (M : EconCSLib.FairDivision.RandomizedDirectFairDivisionMechanism Agent Item) →
  (values reports : EconCSLib.FairDivision.FairDivisionReport Agent Item) →
  (i : Agent) → EconCSLib.pmfExp (M.allocationLaw reports) fun A => values i (A i)
```

Exact Lean library declaration

```
noncomputable def expectedUtility
  [Fintype (Allocation Agent Item)] [DecidableEq (Allocation Agent Item)]
  (M : RandomizedDirectFairDivisionMechanism Agent Item)
  (values reports : FairDivisionReport Agent Item) (i : Agent) : ℝ :=
  EconCSLib.pmfExp (M.allocationLaw reports)
  (fun A => values i (A i))
```

Recorded source-to-library screening Verdict: not recorded

Reviewer check: ☐ Matches source input

If left unchecked, explain the discrepancy or uncertainty below.

Reviewer annotation

EconCSLib.FairDivision.RandomizedDirectFairDivisionMechanism.Truthful

Source connection: no explicit byte-pinned paper source connection is registered.

Lean-expanded library semantic target

```
∀ {Agent : Type u_1} {Item : Type u_2} [inst : Fintype (EconCSLib.FairDivision.Allocation Agent Item)]
  [inst_1 : DecidableEq (EconCSLib.FairDivision.Allocation Agent Item)] [inst_2 : DecidableEq Agent]
  (M : EconCSLib.FairDivision.RandomizedDirectFairDivisionMechanism Agent Item)
  (values : EconCSLib.FairDivision.FairDivisionReport Agent Item) (i : Agent)
  (report : EconCSLib.FairDivision.Bundle Item → ℝ),
  M.expectedUtility values (Function.update values i report) i ≤ M.expectedUtility values values i
```

Exact Lean library declaration

```
def Truthful
  [Fintype (Allocation Agent Item)] [DecidableEq (Allocation Agent Item)]
  [DecidableEq Agent]
  (M : RandomizedDirectFairDivisionMechanism Agent Item) : Prop :=
  ∀ (values : FairDivisionReport Agent Item)
    (i : Agent) (report : Bundle Item → ℝ),
    M.expectedUtility values (Function.update values i report) i ≤
      M.expectedUtility values values i
```

Recorded source-to-library screening Verdict: not recorded

Reviewer check: ☐ Matches source input

If left unchecked, explain the discrepancy or uncertainty below.

Reviewer annotation

EconCSLib.FairDivision.RandomizedDirectFairDivisionMechanism.mk

Source connection: no explicit byte-pinned paper source connection is registered.

Lean declaration type (metadata)

```
{Agent : Type u_3} →  
  {Item : Type u_4} →  
    (EconCSLib.FairDivision.FairDivisionReport Agent Item → PMF (EconCSLib.FairDivision.Allocation Agent Item)) →  
      EconCSLib.FairDivision.RandomizedDirectFairDivisionMechanism Agent Item
```

Exact Lean library declaration

library declaration is not registered or discoverable

Recorded source-to-library screening Verdict: not recorded

Reviewer check: ☐ Matches source input

If left unchecked, explain the discrepancy or uncertainty below.

Reviewer annotation

EconCSLib.FairDivision.ReportEnvyFree

Source connection: no explicit byte-pinned paper source connection is registered.

Lean-expanded library semantic target

```
∀ {Agent : Type u_1} {Item : Type u_2} (values : EconCSLib.FairDivision.FairDivisionReport Agent Item)
  (A : EconCSLib.FairDivision.Allocation Agent Item) (i j : Agent), values i (A j) ≤ values i (A i)
```

Exact Lean library declaration

```
def ReportEnvyFree (values : FairDivisionReport Agent Item)
  (A : Allocation Agent Item) : Prop :=
  ∀ i j, values i (A j) ≤ values i (A i)
```

Recorded source-to-library screening Verdict: not recorded

Reviewer check: ☐ Matches source input

If left unchecked, explain the discrepancy or uncertainty below.

Reviewer annotation

EconCSLib.FairDivision.ReturnsAllocationOf

Source connection: no explicit byte-pinned paper source connection is registered.

Lean-expanded library semantic target

```
∀ {Agent : Type u_1} {Item : Type u_2} [inst : DecidableEq Item]
  (M : EconCSLib.FairDivision.DirectFairDivisionMechanism Agent Item) (goods : Finset Item)
  (values : EconCSLib.FairDivision.FairDivisionReport Agent Item),
  EconCSLib.FairDivision.IsAllocationOf (M.allocation values) goods
```

Exact Lean library declaration

```
def ReturnsAllocationOf [DecidableEq Item]
  (M : DirectFairDivisionMechanism Agent Item) (goods : Finset Item) : Prop :=
  ∀ values : FairDivisionReport Agent Item,
  IsAllocationOf (M.allocation values) goods
```

Recorded source-to-library screening Verdict: not recorded

Reviewer check: ☐ Matches source input

If left unchecked, explain the discrepancy or uncertainty below.

Reviewer annotation

EconCSLib.FairDivision>ReturnsEnvyFreeWheneverExists

Source connection: no explicit byte-pinned paper source connection is registered.

Lean-expanded library semantic target

```
∀ {Agent : Type u_1} {Item : Type u_2} [inst : DecidableEq Item]
(M : EconCSLib.FairDivision.DirectFairDivisionMechanism Agent Item) (goods : Finset Item)
(values : EconCSLib.FairDivision.FairDivisionReport Agent Item),
(∃ A, EconCSLib.FairDivision.IsAllocationOf A goods ∧ EconCSLib.FairDivision.ReportEnvyFree values A) →
EconCSLib.FairDivision.ReportEnvyFree values (M.allocation values)
```

Exact Lean library declaration

```
def ReturnsEnvyFreeWheneverExists [DecidableEq Item]
(M : DirectFairDivisionMechanism Agent Item) (goods : Finset Item) : Prop :=
  ∀ values : FairDivisionReport Agent Item,
    (∃ A : Allocation Agent Item, IsAllocationOf A goods ∧ ReportEnvyFree values A) →
      ReportEnvyFree values (M.allocation values)
```

Recorded source-to-library screening Verdict: not recorded

Reviewer check: ☐ Matches source input

If left unchecked, explain the discrepancy or uncertainty below.

Reviewer annotation

EconCSLib.FairDivision>ReturnsMinimumReportEnvy

Source connection: no explicit byte-pinned paper source connection is registered.

Lean-expanded library semantic target

```
∀ {Agent : Type u_1} {Item : Type u_2} [inst : Fintype Agent] [inst_1 : Nonempty Agent] [inst_2 : DecidableEq Item]
(M : EconCSLib.FairDivision.DirectFairDivisionMechanism Agent Item) (goods : Finset Item)
(values : EconCSLib.FairDivision.FairDivisionReport Agent Item),
EconCSLib.FairDivision.MinimumReportEnvyAllocation values goods (M.allocation values)
```

Exact Lean library declaration

```
def ReturnsMinimumReportEnvy [Fintype Agent] [Nonempty Agent] [DecidableEq Item]
(M : DirectFairDivisionMechanism Agent Item) (goods : Finset Item) : Prop :=
  ∀ values : FairDivisionReport Agent Item,
    MinimumReportEnvyAllocation values goods (M.allocation values)
```

Recorded source-to-library screening Verdict: not recorded

Reviewer check: ☐ Matches source input

If left unchecked, explain the discrepancy or uncertainty below.

Reviewer annotation

EconCSLib.FairDivision.addItem

Source connection: no explicit byte-pinned paper source connection is registered.

Lean-expanded library semantic target

```
{Agent : Type u_1} →  
{Item : Type u_2} →  
  [inst : DecidableEq Agent] →  
  [inst_1 : DecidableEq Item] →  
    (A : EconCSLib.FairDivision.Allocation Agent Item) →  
      (owner : Agent) → (g : Item) → (a : Agent) → if a = owner then insert g (A a) else A a
```

Exact Lean library declaration

```
def addItem [DecidableEq Agent] [DecidableEq Item]  
  (A : Allocation Agent Item) (owner : Agent) (g : Item) :  
  Allocation Agent Item :=  
  fun i => if i = owner then insert g (A i) else A i
```

Recorded source-to-library screening Verdict: not recorded

Reviewer check: ☐ Matches source input

If left unchecked, explain the discrepancy or uncertainty below.

Reviewer annotation

EconCSLib.FairDivision.aggregateMeasure

Source connection: no explicit byte-pinned paper source connection is registered.

Lean-expanded library semantic target

```
{Agent : Type u_1} →  
{Ω : Type u_2} →  
  [inst : MeasurableSpace Ω] → [inst_1 : Fintype Agent] → (μ : Agent → MeasureTheory.Measure Ω) → ∑ agent, μ agent
```

Exact Lean library declaration

```
noncomputable def aggregateMeasure [Fintype Agent]  
  (μ : Agent → Measure Ω) : Measure Ω :=  
  ∑ agent : Agent, μ agent
```

Recorded source-to-library screening Verdict: not recorded

Reviewer check: ☐ Matches source input

If left unchecked, explain the discrepancy or uncertainty below.

Reviewer annotation

EconCSLib.FairDivision.envyBoundedBy_addItem_of_unenvied

Source connection: no explicit byte-pinned paper source connection is registered.

Lean declaration type (metadata)

```
∀ {Agent : Type u_1} {Item : Type u_2} [inst : DecidableEq Agent] [inst_1 : DecidableEq Item]
  (v : EconCSLib.FairDivision.Valuation Agent Item) (A : EconCSLib.FairDivision.Allocation Agent Item) (owner : Agent)
  (g : Item) {α : ℝ},
  0 ≤ α →
  EconCSLib.FairDivision.EnvyBoundedBy v A α →
  EconCSLib.FairDivision.MarginalBound v α →
  EconCSLib.FairDivision.NoOneEnvies v A owner →
  EconCSLib.FairDivision.EnvyBoundedBy v (EconCSLib.FairDivision.addItem A owner g) α
```

Exact Lean library declaration

```
theorem envyBoundedBy_addItem_of_unenvied [DecidableEq Agent] [DecidableEq Item]
  (v : Valuation Agent Item) (A : Allocation Agent Item)
  (owner : Agent) (g : Item) {α : ℝ}
  (hnonneg : 0 ≤ α)
  (hbound : EnvyBoundedBy v A α)
  (hmargin : MarginalBound v α)
  (hunenvied : NoOneEnvies v A owner) :
  EnvyBoundedBy v (addItem A owner g) α
```

Recorded source-to-library screening Verdict: not recorded

Reviewer check: ☐ Matches source input

If left unchecked, explain the discrepancy or uncertainty below.

Reviewer annotation

EconCSLib.FairDivision.isAllocationOf_addItem_insert

Source connection: no explicit byte-pinned paper source connection is registered.

Lean declaration type (metadata)

```
∀ {Agent : Type u_1} {Item : Type u_2} [inst : DecidableEq Agent] [inst_1 : DecidableEq Item]
  (A : EconCSLib.FairDivision.Allocation Agent Item) (goods : Finset Item) (owner : Agent) (g : Item),
  EconCSLib.FairDivision.IsAllocationOf A goods →
  g ∉ goods → EconCSLib.FairDivision.IsAllocationOf (EconCSLib.FairDivision.addItem A owner g) (insert g goods)
```

Exact Lean library declaration

```
theorem isAllocationOf_addItem_insert [DecidableEq Agent] [DecidableEq Item]
  (A : Allocation Agent Item) (goods : Finset Item) (owner : Agent) (g : Item)
  (halloc : IsAllocationOf A goods)
  (hg : g ∉ goods) :
  IsAllocationOf (addItem A owner g) (insert g goods)
```

Recorded source-to-library screening Verdict: not recorded

Reviewer check: ☐ Matches source input

If left unchecked, explain the discrepancy or uncertainty below.

Reviewer annotation

EconCSLib.FairDivision.boundedEnvyStep

Source connection: no explicit byte-pinned paper source connection is registered.

Lean-expanded library semantic target

```
{Agent : Type u_1} →
{Item : Type u_2} →
[inst : Finite Agent] →
[inst_1 : Finite Item] →
[inst_2 : DecidableEq Agent] →
[inst_3 : DecidableEq Item] →
[inst_4 : Nonempty Agent] →
(v : EconCSLib.FairDivision.Valuation Agent Item) →
{α : ℝ} →
(hanonneg : 0 ≤ α) →
(hmargin : EconCSLib.FairDivision.MarginalBound v α) →
(g : Item) →
(state :
  (goods : Finset Item) ×
  { A //
    EconCSLib.FairDivision.IsAllocationOf A goods ∧
    EconCSLib.FairDivision.EnvyBoundedBy v A α } ) →
let goods := state.fst;
let A := ↑state.snd;
have halloc := ⋯;
have hbound := ⋯;
if hg : g ∈ goods then state
else
  have hreduce := ⋯;
  have reduction_exists := ⋯;
  let B := Classical.choose reduction_exists;
  have owner_exists := ⋯;
  let owner := Classical.choose owner_exists;
  have B_props := ⋯;
  have hBalloc := ⋯;
  have hBbound := ⋯;
  have hunenvied := ⋯;
  let A_new := EconCSLib.FairDivision.addItem B owner g;
  let goods_new := insert g goods;
  have halloc_new := ⋯;
  have hbound_new := ⋯;
  (goods_new, (A_new, ⋯))
```

Exact Lean library declaration

```
noncomputable def boundedEnvyStep (hanonneg : 0 ≤ α) (hmargin : MarginalBound v α)
  (g : Item) (state : Σ (goods : Finset Item), { A : Allocation Agent Item // IsAllocationOf A goods ∧ EnvyBoundedBy v A α }) :
  Σ (goods : Finset Item), { A : Allocation Agent Item // IsAllocationOf A goods ∧ EnvyBoundedBy v A α } :=
let goods := state.1
let A := state.2.val
let halloc := state.2.property.1
let hbound := state.2.property.2
if hg : g ∈ goods then state else
have hreduce := hasUnenviedReduction_of_acyclicReduction v (hasAcyclicReduction_of_envyCycleListExtraction v
↔ (hasEnvyCycleListExtraction_of_finite v))
let reduction_exists := hreduce goods A halloc hbound
let B := Classical.choose reduction_exists
let owner_exists := Classical.choose_spec reduction_exists
let owner := Classical.choose owner_exists
let B_props := Classical.choose_spec owner_exists
let hBalloc := B_props.1
let hBbound := B_props.2.1
let hunenvied := B_props.2.2
let A_new := addItem B owner g
let goods_new := insert g goods
have halloc_new : IsAllocationOf A_new goods_new := by
  classical
  exact isAllocationOf_addItem_insert B goods owner g hBalloc hg
have hbound_new : EnvyBoundedBy v A_new α := by
  classical
  exact envyBoundedBy_addItem_of_unenvied v B owner g hanonneg hBbound hmargin hunenvied
(goods_new, (A_new, halloc_new, hbound_new))
```

Recorded source-to-library screening Verdict: not recorded

Reviewer check: ☐ Matches source input

If left unchecked, explain the discrepancy or uncertainty below.

Reviewer annotation

EconCSLib.FairDivision.emptyAllocation

Source connection: no explicit byte-pinned paper source connection is registered.

Lean-expanded library semantic target

(Agent : Type u_1) → (Item : Type u_2) → Agent → ∅

Exact Lean library declaration

```
def emptyAllocation (Agent Item : Type*) : Allocation Agent Item :=  
  fun _ => ∅
```

Recorded source-to-library screening Verdict: not recorded

Reviewer check: ☐ Matches source input

If left unchecked, explain the discrepancy or uncertainty below.

Reviewer annotation

EconCSLib.FairDivision.boundedEnvyAlgorithm

Source connection: no explicit byte-pinned paper source connection is registered.

Lean-expanded library semantic target

```
{Agent : Type u_1} →
{Item : Type u_2} →
[inst : Finite Agent] →
[inst_1 : Finite Item] →
[inst_2 : DecidableEq Agent] →
[inst_3 : DecidableEq Item] →
[inst_4 : Nonempty Agent] →
(v : EconCSLib.FairDivision.Valuation Agent Item) →
{α : ℝ} →
(hanonneg : 0 ≤ α) →
(hmargin : EconCSLib.FairDivision.MarginalBound v α) →
(goodsList : List Item) →
List.foldr (EconCSLib.FairDivision.boundedEnvyStep v hanonneg hmargin)
(∅, (EconCSLib.FairDivision.emptyAllocation Agent Item, ...)) goodsList
```

Exact Lean library declaration

```
noncomputable def boundedEnvyAlgorithm (hanonneg : 0 ≤ α) (hmargin : MarginalBound v α)
(goodsList : List Item) :
Σ (goods : Finset Item), { A : Allocation Agent Item // IsAllocationOf A goods ∧ EnvyBoundedBy v A α } :=
goodsList.foldr (boundedEnvyStep v hanonneg hmargin)
(∅, (emptyAllocation Agent Item, isAllocationOf_empty, by
intro i j
simp [emptyAllocation, envy, hanonneg]))
```

Recorded source-to-library screening Verdict: not recorded

Reviewer check: ☐ Matches source input

If left unchecked, explain the discrepancy or uncertainty below.

Reviewer annotation

EconCSLib.FairDivision.finiteSingletonsWithResidualPieceSet

Source connection: no explicit byte-pinned paper source connection is registered.

Lean-expanded library semantic target

```
∀ {Ω : Type u_2} {Residual : Type u_4} (H : Finset Ω) (residualSet : Residual → Set Ω) (a : 1H ⊗ Residual) (a_1 : Ω),
  (match a with
  | Sum.inl x => {↑x}
  | Sum.inr r => residualSet r)
  a_1
```

Exact Lean library declaration

```
def finiteSingletonsWithResidualPieceSet
  (H : Finset Ω) (residualSet : Residual → Set Ω) :
  ({x : Ω // x ∈ H} ⊗ Residual) → Set Ω
  | Sum.inl x => {x.1}
  | Sum.inr r => residualSet r

/-- A finite aggregate measure has only finitely many point masses above `α`. -/
theorem highAggregatePointMasses_finite
  [MeasurableSingletonClass Ω] [Fintype Agent]
  (μ : Agent → Measure Ω) [∀ agent, IsFiniteMeasure (μ agent)]
  {α : ℝ} (hα : 0 < α) :
  (highAggregatePointMasses μ α).Finite := by
haveI : IsFiniteMeasure (aggregateMeasure μ) := by
  unfold aggregateMeasure
  infer_instance
exact
  Set.Finite.subset
    (EconCSLib.finite_setOf_le_measureReal_singleton
      (μ := aggregateMeasure μ) hα)
    (by
      intro x hx
      exact le_of_lt (by simpa [highAggregatePointMasses] using hx))
```

Recorded source-to-library screening Verdict: not recorded

Reviewer check: ☐ Matches source input

If left unchecked, explain the discrepancy or uncertainty below.

Reviewer annotation

EconCSLib.FairDivision.maxMarginal

Source connection: no explicit byte-pinned paper source connection is registered.

Lean-expanded library semantic target

```
{Agent : Type u_1} →
{Item : Type u_2} →
  [inst : Fintype Agent] →
    [inst_1 : Fintype Item] →
      [inst_2 : DecidableEq Item] →
        [inst_3 : Nonempty Agent] →
          [inst_4 : Nonempty Item] →
            (v : EconCSLib.FairDivision.Valuation Agent Item) →
              (Finset.univ.product (Finset.univ.powerset.product Finset.univ)).sup' ... fun x =>
                v.value x.1 (insert x.2.2 x.2.1) - v.value x.1 x.2.1
```

Exact Lean library declaration

```
noncomputable def maxMarginal
  [Fintype Agent] [Fintype Item] [DecidableEq Item]
  [Nonempty Agent] [Nonempty Item]
  (v : Valuation Agent Item) : ℝ :=
  ((Finset.univ : Finset Agent).product
    (((Finset.univ : Finset Item).powerset).product
      (Finset.univ : Finset Item))).sup'
  (by
    obtain ⟨agent⟩ := (inferInstance : Nonempty Agent)
    obtain ⟨g⟩ := (inferInstance : Nonempty Item)
    exact ⟨(agent, (∅, g)), by simp⟩)
  (fun x => v.value x.1 (insert x.2.2 x.2.1) - v.value x.1 x.2.1)
```

Recorded source-to-library screening Verdict: not recorded

Reviewer check: ☐ Matches source input

If left unchecked, explain the discrepancy or uncertainty below.

Reviewer annotation

EconCSLib.FairDivision.measurePartitionCertificateOfFiniteSingletonsAndResidualAggregate

Source connection: no explicit byte-pinned paper source connection is registered.

Lean-expanded library semantic target

```
{Agent : Type u_1} →
{Ω : Type u_2} →
{Residual : Type u_4} →
[inst : MeasurableSpace Ω] →
[inst_1 : MeasurableSingletonClass Ω] →
[inst_2 : DecidableEq Ω] →
[inst_3 : Fintype Agent] →
(μ : Agent → MeasureTheory.Measure Ω) →
[inst_4 : ∀ (agent : Agent), MeasureTheory.IsFiniteMeasure (μ agent)] →
{α : ℝ} →
(H : Finset Ω) →
(residualSet : Residual → Set Ω) →
(hmeas_residual : ∀ (piece : Residual), MeasurableSet (residualSet piece)) →
(hdisjoint_residual : Set.univ.PairwiseDisjoint residualSet) →
(hcover_residual : Set.iUnion residualSet = (↑H)c) →
(hsingleton : ∀ (agent : Agent), ∀ x ∈ H, (μ agent).real {x} ≤ α) →
(haggregate_residual :
  ∀ (piece : Residual),
    (EconCSLib.FairDivision.aggregateMeasure μ).real (residualSet piece) ≤ α) →
{
  pieceSet :=
    EconCSLib.FairDivision.finiteSingletonsWithResidualPieceSet H residualSet,
  measurable_piece := ⋯, pairwiseDisjoint := ⋯, cover := ⋯, piece_value_le := ⋯ }
}
```

Exact Lean library declaration

noncomputable def measurePartitionCertificateOfFiniteSingletonsAndResidualAggregate

```
[MeasurableSingletonClass Ω] [DecidableEq Ω]
[Fintype Agent] (μ : Agent → Measure Ω)
[∀ agent, IsFiniteMeasure (μ agent)]
{α : ℝ} (H : Finset Ω) (residualSet : Residual → Set Ω)
(hmeas_residual : ∀ piece, MeasurableSet (residualSet piece))
(hdisjoint_residual : Set.PairwiseDisjoint Set.univ residualSet)
(hcover_residual : Set.iUnion residualSet = (H : Set Ω)c)
(hsingleton : ∀ agent x, x ∈ H → (μ agent).real ({x} : Set Ω) ≤ α)
(haggregate_residual :
  ∀ piece, (aggregateMeasure μ).real (residualSet piece) ≤ α) :
  MeasurePartitionCertificate μ α ({x : Ω // x ∈ H} ⊗ Residual) where
pieceSet := finiteSingletonsWithResidualPieceSet H residualSet
measurable_piece := by
  intro piece
  cases piece with
  | inl x => exact MeasurableSet.singleton x.1
  | inr r => exact hmeas_residual r
pairwiseDisjoint := by
  intro piece _ piece' _ hne
  have hres_subset : ∀ r, residualSet r ⊆ (H : Set Ω)c := by
    intro r y hy
    have hy_union : y ∈ Set.iUnion residualSet := by
      exact Set.mem_iUnion.mpr (r, hy)
    simp [hcover_residual] using hy_union
  cases piece with
  | inl x =>
    cases piece' with
    | inl y =>
      have hxy : x.1 ≠ y.1 := by
        intro hval
        exact hne (by cases x; cases y; simp at hval ⊢; exact hval)
      exact Set.disjoint_singleton.2 hxy
    | inr r =>
      exact Set.disjoint_singleton_left.2
        (by
          intro hxmem
          exact (hres_subset r hxmem) x.2)
  | inr r =>
    cases piece' with
    | inl x =>
      exact Set.disjoint_singleton_right.2
        (by
          intro hxmem
          exact (hres_subset r hxmem) x.2)
    | inr r' =>
      have hrr' : r ≠ r' := by
        intro h
        exact hne (by simp [h])
      exact hdisjoint_residual (by simp) (by simp) hrr'
cover := by
  ext y
  constructor
  · intro _
    simp
  · intro _
    by_cases hyH : y ∈ H
    · exact Set.mem_iUnion.mpr
      (Sum.inl (y, hyH), by simp [finiteSingletonsWithResidualPieceSet])
    · have hy_residual : y ∈ Set.iUnion residualSet := by
      simp [hcover_residual] using hyH
      rcases Set.mem_iUnion.mp hy_residual with (r, hyr)
```

```

    exact Set.mem_iUnion.mpr
    (Sum.inr r, by simp [finiteSingletonsWithResidualPieceSet] using hyr)
piece_value_le := by
  intro agent piece
  cases piece with
  | inl x => exact hsingleton agent x.1 x.2
  | inr r =>
    exact (measureReal_le_aggregateMeasureReal  $\mu$  agent (residualSet r)).trans
      (haggregate_residual r)

```

Recorded source-to-library screening Verdict: not recorded

Reviewer check: ☐ Matches source input

If left unchecked, explain the discrepancy or uncertainty below.

Reviewer annotation

EconCSLib.Math.TendsToZero

Source connection: no explicit byte-pinned paper source connection is registered.

Lean-expanded library semantic target

$\forall (\varepsilon : \mathbb{N} \rightarrow \mathbb{R}), \text{Filter.Tendsto } \varepsilon \text{ Filter.atTop (nhds } 0)$

Exact Lean library declaration

```
def TendsToZero ( $\varepsilon : \mathbb{N} \rightarrow \mathbb{R}$ ) : Prop :=  
  Tendsto  $\varepsilon$  atTop (nhds 0)
```

Recorded source-to-library screening Verdict: not recorded

Reviewer check: ☐ Matches source input

If left unchecked, explain the discrepancy or uncertainty below.

Reviewer annotation

EconCSLib.Probability.RealIntervalPartition

Source connection: no explicit byte-pinned paper source connection is registered.

Lean declaration type (metadata)

MeasureTheory.Measure $\mathbb{R} \rightarrow \mathbb{R} \rightarrow \mathbb{R} \rightarrow \mathbb{R} \rightarrow \text{Type } 1$

Exact Lean library declaration

```
structure RealIntervalPartition (μ : Measure ℝ) (α a b : ℝ) where
  Piece : Type
  instFintype : Fintype Piece
  instDecidableEq : DecidableEq Piece
  pieceSet : Piece → Set ℝ
  measurable_piece : ∀ piece, MeasurableSet (pieceSet piece)
  pairwiseDisjoint : Set.PairwiseDisjoint Set.univ pieceSet
  cover : Set.iUnion pieceSet = Ioc a b
  piece_mass_le : ∀ piece, μ.real (pieceSet piece) ≤ α
```

Recorded source-to-library screening Verdict: not recorded

Reviewer check: ☐ Matches source input

If left unchecked, explain the discrepancy or uncertainty below.

Reviewer annotation

EconCSLib.Probability.RealIntervalPartition.Piece

Source connection: no explicit byte-pinned paper source connection is registered.

Lean-expanded library semantic target

```
{μ : MeasureTheory.Measure ℝ} → {α a b : ℝ} → (self : EconCSLib.Probability.RealIntervalPartition μ α a b) → self.1
```

Exact Lean library declaration

library declaration is not registered or discoverable

Recorded source-to-library screening Verdict: not recorded

Reviewer check: ☐ Matches source input

If left unchecked, explain the discrepancy or uncertainty below.

Reviewer annotation

EconCSLib.Probability.RealIntervalPartition.instDecidableEq

Source connection: no explicit byte-pinned paper source connection is registered.

Lean-expanded library semantic target

```
{μ : MeasureTheory.Measure ℝ} →  
{α a b : ℝ} → (self : EconCSLib.Probability.RealIntervalPartition μ α a b) → (a_1 b : self.Piece) → self.3 a_1 b
```

Exact Lean library declaration

library declaration is not registered or discoverable

Recorded source-to-library screening Verdict: not recorded

Reviewer check: ☐ Matches source input

If left unchecked, explain the discrepancy or uncertainty below.

Reviewer annotation

EconCSLib.Probability.RealIntervalPartition.instFintype

Source connection: no explicit byte-pinned paper source connection is registered.

Lean-expanded library semantic target

$\{\mu : \text{MeasureTheory.Measure } \mathbb{R}\} \rightarrow \{\alpha \text{ a b} : \mathbb{R}\} \rightarrow (\text{self} : \text{EconCSLib.Probability.RealIntervalPartition } \mu \ \alpha \text{ a b}) \rightarrow \text{self}.2$

Exact Lean library declaration

library declaration is not registered or discoverable

Recorded source-to-library screening Verdict: not recorded

Reviewer check: ☐ Matches source input

If left unchecked, explain the discrepancy or uncertainty below.

Reviewer annotation

EconCSLib.Probability.RealIntervalPartition.pieceSet

Source connection: no explicit byte-pinned paper source connection is registered.

Lean-expanded library semantic target

```
∀ {μ : MeasureTheory.Measure ℝ} {α a b : ℝ} (self : EconCSLib.Probability.RealIntervalPartition μ α a b)
  (a : self.Piece) (a_1 : ℝ), self.4 a a_1
```

Exact Lean library declaration

library declaration is not registered or discoverable

Recorded source-to-library screening Verdict: not recorded

Reviewer check: ☐ Matches source input

If left unchecked, explain the discrepancy or uncertainty below.

Reviewer annotation

EconCSLib.pmfProb

Source connection: no explicit byte-pinned paper source connection is registered.

Lean-expanded library semantic target

```
{α : Type u_1} →  
  [inst : Fintype α] →  
    [inst_1 : DecidableEq α] →  
      (μ : PMF α) → (p : α → Prop) → [inst_2 : DecidablePred p] → EconCSLib.pmfExp μ fun a => if p a then 1 else 0
```

Exact Lean library declaration

```
noncomputable def pmfProb {α : Type*} [Fintype α] [DecidableEq α]  
  (μ : PMF α) (p : α → Prop) [DecidablePred p] : ℝ :=  
  pmfExp μ (fun a => if p a then 1 else 0)
```

Recorded source-to-library screening Verdict: not recorded

Reviewer check: ☐ Matches source input

If left unchecked, explain the discrepancy or uncertainty below.

Reviewer annotation

EconCSLib.pmfProduct

Source connection: no explicit byte-pinned paper source connection is registered.

Lean-expanded library semantic target

```
(ι : Type u_1) →  
(α : Type u_2) →  
  [inst : Fintype ι] →  
    [inst_1 : DecidableEq ι] → [inst_2 : Fintype α] → (μ : PMF α) → PMF.ofFintype (fun f => [] i, μ (f i)) ...
```

Exact Lean library declaration

```
noncomputable def pmfProduct (ι α : Type*) [Fintype ι] [DecidableEq ι] [Fintype α]  
  (μ : PMF α) : PMF (ι → α) :=  
  PMF.ofFintype (fun f : ι → α => [] i : ι, μ (f i)) (by  
    classical  
    have hcoord : ∑ a : α, μ a = 1 := by  
      rw [← PMF.tsum_coe μ, tsum_fintype]  
    calc  
      ∑ f : ι → α, [] i : ι, μ (f i)  
      = [] i : ι, ∑ a : α, μ a := by  
        symm  
        simp using  
          (Finset.prod_univ_sum  
            (t := fun _i : ι => (Finset.univ : Finset α))  
            (f := fun _i a => μ a))  
      _ = 1 := by simp [hcoord])
```

Recorded source-to-library screening Verdict: not recorded

Reviewer check: ☐ Matches source input

If left unchecked, explain the discrepancy or uncertainty below.

Reviewer annotation

EconCSLib.uniformPMF

Source connection: no explicit byte-pinned paper source connection is registered.

Lean-expanded library semantic target

$(\alpha : \text{Type } u_1) \rightarrow [\text{inst} : \text{Fintype } \alpha] \rightarrow [\text{inst}_1 : \text{Nonempty } \alpha] \rightarrow \text{PMF.ofFintype } (\text{fun } x \Rightarrow (\uparrow(\text{Fintype.card } \alpha))^{-1}) \dots$

Exact Lean library declaration

```
noncomputable def uniformPMF (α : Type*) [Fintype α] [Nonempty α] : PMF α :=
  PMF.ofFintype (fun _ : α => ((Fintype.card α : ENNReal)⁻¹)) (by
    rw [Finset.sum_const, Finset.card_univ]
    have hcard : (Fintype.card α : ENNReal) ≠ 0 := by
      exact_mod_cast (Fintype.card_pos_iff.mpr <Nonempty α>.ne')
    simp [nsmul_eq_mul] using
      (ENNReal.mul_inv_cancel hcard (ENNReal.natCast_ne_top (Fintype.card α))))
```

Recorded source-to-library screening Verdict: not recorded

Reviewer check: ☐ Matches source input

If left unchecked, explain the discrepancy or uncertainty below.

Reviewer annotation

Source claims

Review row 1

Source locator: cited publication, lines 141-145

Verbatim source input

the envy of p for q :
 $epq(A) = \max\{0, v_p(A_q) - v_p(A_p)\}$.
We define $e(A)$ to be the maximum envy between any pair
of players.
 $e(A) = \max\{epq(A), p, q \in N\}$.

Expanded PaperInterface specification

```
∀ {Agent : Type u_1} {Item : Type u_2} (v : EconCSLib.FairDivision.Valuation Agent Item)
  (A : EconCSLib.FairDivision.Allocation Agent Item) (i j : Agent),
  LMMS04FairDivision.PaperInterface.envy v A i j = max 0 (v.value i (A j) - v.value i (A i))
```

Lean proof endpoint (built separately)

LMMS04FairDivision.PaperInterface.envy

Recorded source-to-Spec screening Verdict: not recorded

Reason: not recorded

Reviewer check: ☐ Matches source input

If left unchecked, explain the discrepancy or uncertainty below.

Reviewer annotation

Review row 2

Source locator: cited publication, lines 141-145

Verbatim source input

the envy of p for q :
$$\text{epq}(A) = \max\{0, v_p(A_q) - v_p(A_p)\}.$$

We define $e(A)$ to be the maximum envy between any pair of players.
$$e(A) = \max\{\text{epq}(A), p, q \in N\}.$$

Expanded PaperInterface specification

```
∀ {Agent : Type u_1} {Item : Type u_2} (v : EconCSLib.FairDivision.Valuation Agent Item)
  (A : EconCSLib.FairDivision.Allocation Agent Item),
  LMMS04FairDivision.PaperInterface.envyFree v A = ∀ (i j : Agent), v.value i (A j) ≤ v.value i (A i)
```

Lean proof endpoint (built separately)

LMMS04FairDivision.PaperInterface.envyFree

Recorded source-to-Spec screening Verdict: not recorded

Reason: not recorded

Reviewer check: ☐ Matches source input

If left unchecked, explain the discrepancy or uncertainty below.

Reviewer annotation

Review row 3

Source locator: cited publication, lines 147-165

Verbatim source input

A natural question is whether there exist allocations with bounded envy. We obtain a bound on the envy in terms of the maximum marginal utility of the goods, α . The marginal utility of a good i with respect to a player p and a subset of goods S , is the amount by which it increases the utility of p , when added to S , i.e., $vp(S \cup \{i\}) - vp(S)$. The maximum marginal utility is:

$$\alpha = \max_{S,p,i} vp(S \cup \{i\}) - vp(S)$$

In addition to proving a bound on the envy, we present an efficient algorithm that computes a desired allocation. For that, we assume that the algorithm can ask an oracle for the utility of a player p for any subset S .

Theorem 2.1. For any set of goods and any set of players, there exists an allocation A such that the maximum envy of A is bounded by the maximum marginal utility of the goods, α . Furthermore, given oracle access for the utility functions of the players, there is an $O(mn^3)$ time algorithm for finding such an allocation.

Expanded PaperInterface specification

```
∀ {Agent : Type u 1} {Item : Type u 2} (v : EconCSLib.FairDivision.Valuation Agent Item)
  (A : EconCSLib.FairDivision.Allocation Agent Item) (alpha : ℝ),
  LMMS04FairDivision.PaperInterface.envyBoundedBy v A alpha =
    ∀ (i j : Agent), LMMS04FairDivision.PaperInterface.envy v A i j ≤ alpha
```

Lean proof endpoint (built separately)

LMMS04FairDivision.PaperInterface.envyBoundedBy

Recorded source-to-Spec screening Verdict: not recorded

Reason: not recorded

Reviewer check: ☐ Matches source input

If left unchecked, explain the discrepancy or uncertainty below.

Reviewer annotation

Review row 4

Source locator: cited publication, lines 147-155

Verbatim source input

A natural question is whether there exist allocations with bounded envy. We obtain a bound on the envy in terms of the maximum marginal utility of the goods, α . The marginal utility of a good i with respect to a player p and a subset of goods S , is the amount by which it increases the utility of p , when added to S , i.e., $vp(S \cup \{i\}) - vp(S)$. The maximum marginal utility is:
$$\alpha = \max_{S,p,i} vp(S \cup \{i\}) - vp(S)$$

Expanded PaperInterface specification

```
∀ {Agent : Type u_1} {Item : Type u_2} [inst : Fintype Agent] [inst_1 : Fintype Item] [inst_2 : DecidableEq Item]
  [inst_3 : Nonempty Agent] [inst_4 : Nonempty Item] (v : EconCSLib.FairDivision.Valuation Agent Item),
  LMMS04FairDivision.PaperInterface.maxMarginal v = LMMS04FairDivision.paper_max_marginal v
```

Lean proof endpoint (built separately)

LMMS04FairDivision.PaperInterface.maxMarginal

Recorded source-to-Spec screening Verdict: not recorded

Reason: not recorded

Reviewer check: ☐ Matches source input

If left unchecked, explain the discrepancy or uncertainty below.

Reviewer annotation

Review row 5

Source locator: cited publication, lines 201-212

Verbatim source input

Let $N = \{1, 2, \dots, n\}$ be a set of players and $M = \{1, 2, \dots, m\}$ be a set of indivisible goods. A utility function v_p is associated with each player p . For $S \subseteq M$, $v_p(S)$ is the happiness player p derives if she obtains the subset S . We assume that v_p is non-negative and monotone i.e. $v_p(S) \leq v_p(T)$ for every $S \subseteq T$ and every p .
An allocation A is a partition of the goods $A = (A_1, A_2, \dots, A_n)$ where $\bigcup_{p=1}^n A_p = M$ and $A_p \cap A_q = \emptyset$ for all $p \neq q$. A_p is the subset allocated to player p . Note that some of the sets A_p may be empty. A partial allocation will be a partition of some subset of M .

Expanded PaperInterface specification

```
∀ {Agent : Type u_1} {Item : Type u_2} [inst : DecidableEq Item] (A : EconCSLib.FairDivision.Allocation Agent Item)
  (goods : Finset Item),
  LMS04FairDivision.PaperInterface.isAllocationOf A goods = EconCSLib.FairDivision.IsAllocationOf A goods
```

Lean proof endpoint (built separately)

LMS04FairDivision.PaperInterface.isAllocationOf

Recorded source-to-Spec screening Verdict: not recorded

Reason: not recorded

Reviewer check: ☐ Matches source input

If left unchecked, explain the discrepancy or uncertainty below.

Reviewer annotation

Review row 6

Source locator: cited publication, lines 166-197

Verbatim source input

Given an allocation A , we define the envy graph of A as follows: every node of the graph represents a player and there is a directed edge from p to q iff p envies q . The proof of Theorem 2.1 is based on the following Lemma:
Lemma 2.2. For any partial allocation A with envy graph G , we can find another partial allocation B with envy graph H such that:
• $e(B) \leq e(A)$
• H is acyclic.
Proof. If G has no directed cycles, we are done. Suppose that $C = p_1 \rightarrow p_2 \rightarrow \dots \rightarrow p_r \rightarrow p_1$ is a directed cycle in G . If $A = \{A_1, \dots, A_n\}$, we can obtain $A_0 = (A_{01}, \dots, A_{0n})$ by re-allocating the goods as follows: $A_{0p} = A_p$ for all $p \in \{p_1, \dots, p_r\}$, and $A_{0p_1} = A_{p_2}$, $A_{0p_2} = A_{p_3}$, $\dots$, $A_{0p_r} = A_{p_1}$. Note that all players evaluate what they have in A_0 at least as much as what they have in A . Therefore it is easy to see that $e(A_0) \leq e(A)$.
We can also show that the number of edges in the envy graph G_0 corresponding to A_0 has decreased. To see this, first note that the set of the edges between pairs of vertices in $N \setminus C$ has not changed. Also every edge of the form $p \rightarrow p_j$ for $p \in N \setminus C$ and $p_j \in C$ has now become the edge $p \rightarrow p_{j-1}$ (or $p \rightarrow p_r$ if $j = 1$) in G_0 and no more edges of this form have been added. The number of edges of the form $p_j \rightarrow p$ has either decreased or remained the same since players in C are strictly happier. Finally for $p_i \in C$ the number of edges from p_i to vertices in C has decreased by at least 1. Thus by repeatedly removing cycles using the above procedure, we will obtain an allocation B with corresponding envy graph H such that $e(B) \leq e(A)$ and H is acyclic. Since the number of edges decreases at every step, the process will terminate.

Expanded PaperInterface specification

```
∀ {Agent : Type u_3} {Item : Type u_4} [Fintype Agent] [Fintype Item] [DecidableEq Agent] [inst : DecidableEq Item]
[Nonempty Agent] [Nonempty Item] (v : EconCSLib.FairDivision.Valuation Agent Item) {alpha : ℝ} (goods : Finset Item)
(A : EconCSLib.FairDivision.Allocation Agent Item),
  LMMS04FairDivision.ProofInterface.isAllocationOf A goods →
    LMMS04FairDivision.ProofInterface.envyBoundedBy v A alpha →
      ∃ B,
        LMMS04FairDivision.ProofInterface.isAllocationOf B goods ∧
        LMMS04FairDivision.ProofInterface.envyBoundedBy v B alpha ∧ EconCSLib.FairDivision.AcyclicEnvyGraph v B
```

Lean proof endpoint (built separately)

LMMS04FairDivision.PaperInterface.lemma2_2_acyclic_reduction

Recorded source-to-Spec screening Verdict: not recorded

Reason: not recorded

Reviewer check: ☐ Matches source input

If left unchecked, explain the discrepancy or uncertainty below.

Reviewer annotation

Review row 7

Source locator: cited publication, lines 161-165

Verbatim source input

Theorem 2.1. For any set of goods and any set of players, there exists an allocation A such that the maximum envy of A is bounded by the maximum marginal utility of the goods, α . Furthermore, given oracle access for the utility functions of the players, there is an $O(mn^3)$ time algorithm for finding such an allocation.

Expanded PaperInterface specification

```
∀ {Agent : Type u_3} {Item : Type u_4} [inst : Fintype Agent] [inst_1 : Fintype Item] [DecidableEq Agent]
  [inst_3 : DecidableEq Item] [inst_4 : Nonempty Agent] [inst_5 : Nonempty Item]
  (v : EconCSLib.FairDivision.Valuation Agent Item) (goods : Finset Item),
  ∃ A,
    LMMS04FairDivision.ProofInterface.isAllocationOf A goods ∧
    LMMS04FairDivision.ProofInterface.envyBoundedBy v A (LMMS04FairDivision.ProofInterface.maxMarginal v)
```

Lean proof endpoint (built separately)

LMMS04FairDivision.PaperInterface.theorem2_1_bounded_envy_allocation_exists

Recorded source-to-Spec screening Verdict: not recorded

Reason: not recorded

Reviewer check: ☐ Matches source input

If left unchecked, explain the discrepancy or uncertainty below.

Reviewer annotation

Review row 8

Source locator: cited publication, lines 161-165

Verbatim source input

Theorem 2.1. For any set of goods and any set of players, there exists an allocation A such that the maximum envy of A is bounded by the maximum marginal utility of the goods, α . Furthermore, given oracle access for the utility functions of the players, there is an $O(mn^3)$ time algorithm for finding such an allocation.

Expanded PaperInterface specification

```
∀ {Agent : Type u_3} {Item : Type u_4} [Fintype Agent] [Fintype Item] [DecidableEq Agent] [inst : DecidableEq Item]
  [Nonempty Agent] [Nonempty Item] (v : EconCSLib.FairDivision.Valuation Agent Item) {alpha : ℝ},
  0 ≤ alpha →
    EconCSLib.FairDivision.MarginalBound v alpha →
      ∀ (goods : Finset Item),
        ∃ A,
          LMS04FairDivision.ProofInterface.isAllocationOf A goods ∧
          LMS04FairDivision.ProofInterface.envyBoundedBy v A alpha
```

Lean proof endpoint (built separately)

LMS04FairDivision.PaperInterface.theorem2_1_alpha_bounded_allocation_exists

Recorded source-to-Spec screening Verdict: not recorded

Reason: not recorded

Reviewer check: ☐ Matches source input

If left unchecked, explain the discrepancy or uncertainty below.

Reviewer annotation

Review row 9

Source locator: cited publication, lines 219-268

Verbatim source input

Proof of Theorem 2.1. We give an algorithm that produces the desired allocation. The algorithm proceeds in m rounds. At each round one more good is allocated to some player.

In the first round, we allocate good 1 to some player arbitrarily. Clearly the maximum envy is at most α . Suppose at the end of round i , the goods $\{1, \dots, i\}$ have been allocated to the players and the maximum envy is at most α . At round $i + 1$, we construct the envy graph corresponding to the current allocation. We use the procedure of Lemma 2.2 to obtain an allocation A in which the maximum envy is at most α and the new envy graph G is acyclic. Since G is acyclic, there is a player $p \in N$ with in-degree 0, which implies that nobody envies p . We then allocate good $i + 1$ to p . Let $B = (B_1, \dots, B_n)$ be the new allocation. For any 2 players q, r with $q, r \neq p$, $\text{eqr}(B) = \text{eqr}(A) \leq \alpha$. For $q \in N \setminus \{p\}$, since $\text{eqp}(A) = 0$ we have:

$$\text{eqp}(B)$$

exists since atoms have value at most α . Set $S_1 = [0, x]$ and consider the interval $(x, 1]$. Again find the minimum value of $y \in (x, 1]$ such that $\forall p ((x, y]) \leq \alpha$ for every p . Set $S_2 = (x, y]$. We can continue in the same manner until we partition the whole interval $[0, 1]$. It is easy to check that the number of disjoint intervals $S_1, S_2, \dots, S_m$ of the partition will be $O(n/\alpha)$.

We can now treat the intervals $S_1, \dots, S_m$ produced in the previous Lemma as indivisible goods and Theorem 2.1 will complete the proof.

3.

Although we can produce an allocation with bounded envy, in many instances the maximum envy can be smaller than α . Therefore we would like to look at the following two optimization problems:

$$= \max\{0, v_q(A_p \cup \{i\}) - v_q(A_q)\} \\ \leq$$

MINIMIZING ENVY AS AN OPTIMIZATION PROBLEM

$\max\{0, \alpha + v_q(A_p) - v_q(A_q)\} \leq \alpha$
Problem 1: Minimum envy
Compute an allocation A that minimizes the envy

We use a simple amortized analysis for the running time of the algorithm. In Lemma 2.2, we keep removing cycles until the envy-graph is acyclic. Finding a cycle and removing it takes at most $O(n^2)$ time and it decreases the number of edges by at least one. Initially the envy graph has no edges. Allocating a good at any round adds at most n edges to the new envy graph. Since every cycle removal decreases the number of edges, the number of times we have to remove a cycle is at most $O(nm)$ and the total running time is $O(mn^3)$.

Expanded PaperInterface specification

```
∀ {Agent : Type u_3} {Item : Type u_4} [inst : Fintype Agent] [inst_1 : Fintype Item] [inst_2 : DecidableEq Agent]
[inst_3 : DecidableEq Item] [inst_4 : Nonempty Agent] [Nonempty Item]
(v : EconCSLib.FairDivision.Valuation Agent Item) {alpha : ℝ} (halpha nonneg : 0 ≤ alpha)
(hbound : EconCSLib.FairDivision.MarginalBound v alpha) (goodsList : List Item),
goodsList.Nodup →
  LMMS04FairDivision.ProofInterface.isAllocationOf
    (LMMS04FairDivision.paper_lmms_algorithm v halpha_nonneg hbound goodsList) goodsList.toFinset ∧
  LMMS04FairDivision.ProofInterface.envyBoundedBy v
    (LMMS04FairDivision.paper_lmms_algorithm v halpha_nonneg hbound goodsList) alpha
```

Lean proof endpoint (built separately)

LMMS04FairDivision.PaperInterface.theorem2_1_algorithm_correct_list_toFinset

Recorded source-to-Spec screening Verdict: not recorded

Reason: not recorded

Reviewer check: ☐ Matches source input

If left unchecked, explain the discrepancy or uncertainty below.

Reviewer annotation

Review row 10

Source locator: cited publication, lines 299-340

Verbatim source input

In [8], a similar model has been defined with the difference that the utility function of every player is additive and we have a combination of divisible and indivisible goods. More formally, in [8] the problem is to partition a measurable space $(\Omega, \mathcal{F})$. Each player has a utility function which is a probability measure ν_p on $(\Omega, \mathcal{F})$ such that for each ν_p the maximum value of an atom is α . A subset $S \subseteq \Omega$ is an atom for ν_p if $\nu_p(S) > 0$ and $\forall E \subset S$, either $\nu_p(E) = 0$ or $\nu_p(E) = \nu_p(S)$. It is shown that there exist allocations with envy at most $O(n^{3/2})$.

We can prove that our result also holds for their model and hence it improves the bound of $O(n^{3/2})$ to α . The idea is that we can partition Ω into indivisible goods of value at most α and then apply Theorem 2.1.

Theorem 2.3. When the utilities of the players are probability measures on $(\Omega, \mathcal{F}) = ([0, 1], \text{Borel sets})$ with atoms of value at most α , there exists a partition $A = (A_1, \dots, A_n)$ of Ω such that $e(A) \leq \alpha$.

Theorem 3.1. Any (deterministic) algorithm that computes an allocation with minimum envy or minimum envyratio requires a number of queries which is exponential in the number of goods in the worst case.

Proof. Since each measure ν_p has atoms of value at most α , this means that for every $x \in [0, 1]$, $\nu_p(\{x\}) \leq \alpha$. The case $\alpha = 0$ corresponds to an infinitely divisible cake and an envy-free allocation always exists [8]. For $\alpha > 0$ we can reduce the problem to allocating indivisible goods of value at most α and then use Theorem 2.1.

Proof. We give an outline of the proof. Suppose $m = 2k$. We consider the following class of utility functions F . A function v is in F if:

$v(S) = 0$ for all S with $|S| < k$.

$v(S) = 1$ for all S with $|S| > k$.

$v(S) = 1 - v(\bar{S})$ for all $|S| = k$.

Now, consider instances (v, ν) in which there are two players with the same utility function v for some $v \in F$. The number of such instances is doubly exponential in k . Since the algorithm asks only a polynomial number of queries, it

Lemma 2.4. The interval $[0, 1]$ can be partitioned in m disjoint sets $S_1, \dots, S_m$ such that $m = O(n/\alpha)$ and $\nu_p(S_j) \leq \alpha$ for every $p = 1, \dots, n$, $j = 1, \dots, m$.

Proof. Find the minimum possible value for $x \in [0, 1]$

Expanded PaperInterface specification

```
∀ {Agent : Type u_3} [inst : Fintype Agent] [DecidableEq Agent] [Nonempty Agent] {mu : Agent → MeasureTheory.Measure ℝ}
[inst_3 : ∀ (agent : Agent), MeasureTheory.IsFiniteMeasure (mu agent)] {alpha a b : ℝ} (halpha_pos : 0 < alpha)
(hatoms : ∀ (agent : Agent), LMMS04FairDivision.Lemma24.PaperAtomsBoundedBy (mu agent) alpha)
(haggregate_support : (EconCSLib.FairDivision.aggregateMeasure mu).real (Set.Ioc a b)c = 0),
∃ H P A,
  EconCSLib.FairDivision.IsAllocationOf A Finset.univ A
  EconCSLib.FairDivision.EnvyBoundedBy
    (EconCSLib.FairDivision.measurePartitionCertificateOfFiniteSingletonsAndResidualAggregate mu H
      (LMMS04FairDivision.Lemma24.realIntervalResidualSet H P) ...).valuation
  A alpha
```

Lean proof endpoint (built separately)

LMMS04FairDivision.PaperInterface.theorem2_3_real_interval_supported_atom_bound

Recorded source-to-Spec screening Verdict: not recorded

Reason: not recorded

Reviewer check: ☐ Matches source input

If left unchecked, explain the discrepancy or uncertainty below.

Reviewer annotation

Review row 11

Source locator: cited publication, lines 317-354

Verbatim source input

Theorem 3.1. Any (deterministic) algorithm that computes an allocation with minimum envy or minimum envyratio requires a number of queries which is exponential in the number of goods in the worst case.

Proof. Since each measure v_p has atoms of value at most α , this means that for every $x \in [0, 1]$, $v_p(\{x\}) \leq \alpha$. The case $\alpha = 0$ corresponds to an infinitely divisible cake and an envy-free allocation always exists [8]. For $\alpha > 0$ we can reduce the problem to allocating indivisible goods of value at most α and then use Theorem 2.1.

Proof. We give an outline of the proof. Suppose $m = 2k$. We consider the following class of utility functions F . A function v is in F if:

$v(S) = 0$ for all S with $|S| < k$.

$v(S) = 1$ for all S with $|S| > k$.

$v(S) = 1 - v(\bar{S})$ for all $|S| = k$

Now, consider instances (v, v) in which there are two players with the same utility function v for some $v \in F$. The number of such instances is doubly exponential in k . Since the algorithm asks only a polynomial number of queries, it

Lemma 2.4. The interval $[0, 1]$ can be partitioned in m disjoint sets $S_1, \dots, S_m$ such that $m = 0(n/\alpha)$ and $v_p(S_j) \leq \alpha$ for every $p = 1, \dots, n, j = 1, \dots, m$

Proof. Find the minimum possible value for $x \in [0, 1]$ such that $v_p([0, x]) \leq \alpha$ for every player p . Such an x

127

[form-feed in source extraction]

can produce only an exponential number of different query sequences. Therefore, there exist two different functions $u, v \in F$ such that the query sequences corresponding to the instances defined by u and v are the same. Consider the instances (u, v) and (v, u) . The algorithm will produce the same query sequences for both instances and therefore it will produce the same allocation for (u, v) and (v, u) . It is easy to see that although for either case, there exists an allocation which is envy free, there is no single allocation that is envy free for both instances.

Expanded PaperInterface specification

```

V {SourceItem : N → Type u_3} [inst : (n : N) → Fintype (SourceItem n)] [inst_1 : (n : N) → DecidableEq (SourceItem n)]
{ k q : N → N } (C : (n : N) → LMMS04FairDivision.Theorem31.LMMS31MiddleComplementPairs (SourceItem n) (k n)),
( V (n : N), Fintype.card (SourceItem n) = 2 * k n ) →
  V (strategy : (n : N) → LMMS04FairDivision.Theorem31.AdaptiveQueryStrategy (Finset (SourceItem n)) (Bool × Bool))
    (output :
      (n : N) →
        LMMS04FairDivision.Theorem31.QueryTranscript (Bool × Bool) (q n) →
          EconCSLib.FairDivision.Allocation LMMS04FairDivision.Theorem31.LMMS31Agent (SourceItem n)),
    (EconCSLib.Math.TendsToZero fun n => ↑(2 * q n) / ↑(Fintype.card (C n).Pair)) →
    (Vf (n : N) in Filter.atTop, 0 < Fintype.card (C n).Pair) →
      Vf (n : N) in Filter.atTop,
        →V (choice1 choice2 : (C n).Pair → Bool),
          EconCSLib.FairDivision.MinimumReportEnvyAllocation
            (LMMS04FairDivision.Theorem31.Lmms31CrossReport ((C n).hardFunctionOfMiddleChoice choice1)
              ((C n).hardFunctionOfMiddleChoice choice2))
            Finset.univ
            (output n
              (LMMS04FairDivision.Theorem31.twoPlayerHardFunctionAdaptiveTranscript (strategy n)
                ((C n).hardFunctionOfMiddleChoice choice1) ((C n).hardFunctionOfMiddleChoice choice2)))

```

Lean proof endpoint (built separately)

LMMS04FairDivision.PaperInterface.theorem3_1_eventually_minimum_envy_lower_bound_from_twoBit_adaptive_queries

Recorded source-to-Spec screening Verdict: not recorded

Reason: not recorded

Reviewer check: ☐ Matches source input

If left unchecked, explain the discrepancy or uncertainty below.

Reviewer annotation

Review row 12

Source locator: cited publication, lines 317-354

Verbatim source input

Theorem 3.1. Any (deterministic) algorithm that computes an allocation with minimum envy or minimum envyratio requires a number of queries which is exponential in the number of goods in the worst case.

Proof. Since each measure v_p has atoms of value at most α , this means that for every $x \in [0, 1]$, $v_p(\{x\}) \leq \alpha$. The case $\alpha = 0$ corresponds to an infinitely divisible cake and an envy-free allocation always exists [8]. For $\alpha > 0$ we can reduce the problem to allocating indivisible goods of value at most α and then use Theorem 2.1.

Proof. We give an outline of the proof. Suppose $m = 2k$. We consider the following class of utility functions F . A function v is in F if:
 $v(S) = 0$ for all S with $|S| < k$.
 $v(S) = 1$ for all S with $|S| > k$.
 $v(S) = 1 - v(\bar{S})$ for all $|S| = k$.

Now, consider instances (v, v) in which there are two players with the same utility function v for some $v \in F$. The number of such instances is doubly exponential in k . Since the algorithm asks only a polynomial number of queries, it

Lemma 2.4. The interval $[0, 1]$ can be partitioned in m disjoint sets $S_1, \dots, S_m$ such that $m = 0(n/\alpha)$ and $v_p(S_j) \leq \alpha$ for every $p = 1, \dots, n$, $j = 1, \dots, m$.

Proof. Find the minimum possible value for $x \in [0, 1]$ such that $v_p([0, x]) \leq \alpha$ for every player p . Such an x

127

[form-feed in source extraction]
can produce only an exponential number of different query sequences. Therefore, there exist two different functions $u, v \in F$ such that the query sequences corresponding to the instances defined by u and v are the same. Consider the instances (u, v) and (v, u) . The algorithm will produce the same query sequences for both instances and therefore it will produce the same allocation for (u, v) and (v, u) . It is easy to see that although for either case, there exists an allocation which is envy free, there is no single allocation that is envy free for both instances.

Expanded PaperInterface specification

```

V {SourceItem : N → Type u_3} [inst : (n : N) → Fintype (SourceItem n)] [inst_1 : (n : N) → DecidableEq (SourceItem n)]
{ k q : N → N } (C : (n : N) → LMMS04FairDivision.Theorem31.LMMS31MiddleComplementPairs (SourceItem n) (k n)),
(V (n : N), Fintype.card (SourceItem n) = 2 * k n) →
  V (strategy : (n : N) → LMMS04FairDivision.Theorem31.AdaptiveQueryStrategy (Finset (SourceItem n)) (Bool × Bool))
    (output :
      (n : N) →
        LMMS04FairDivision.Theorem31.QueryTranscript (Bool × Bool) (q n) →
          EconCSLib.FairDivision.Allocation LMMS04FairDivision.Theorem31.LMMS31Agent (SourceItem n)),
    (EconCSLib.Math.TendsToZero fun n => ↑(2 * q n) / ↑(Fintype.card (C n).Pair)) →
    (Vf (n : N) in Filter.atTop, 0 < Fintype.card (C n).Pair) →
      Vf (n : N) in Filter.atTop,
        →V (choice1 choice2 : (C n).Pair → Bool),
          LMMS04FairDivision.Theorem31.MinimumReportEnvyRatioAllocation
            (LMMS04FairDivision.Theorem31.Lmms31CrossReport ((C n).hardFunctionOfMiddleChoice choice1)
              ((C n).hardFunctionOfMiddleChoice choice2))
            Finset.univ
            (output n
              (LMMS04FairDivision.Theorem31.twoPlayerHardFunctionAdaptiveTranscript (strategy n)
                ((C n).hardFunctionOfMiddleChoice choice1) ((C n).hardFunctionOfMiddleChoice choice2)))

```

Lean proof endpoint (built separately)

LMMS04FairDivision.PaperInterface.theorem3_1_eventually_minimum_envy_ratio_lower_bound_from_twoBit_adaptive_queries

Recorded source-to-Spec screening Verdict: not recorded

Reason: not recorded

Reviewer check: ☐ Matches source input

If left unchecked, explain the discrepancy or uncertainty below.

Reviewer annotation

Review row 13

Source locator: cited publication, lines 356-365

Verbatim source input

player whose current bundle has the least value. In [5] it was proved (in the context of scheduling) that the approximation factor of Graham’s algorithm is 1.4 for the ratio problem. Theorem 3.2. [5] Graham’s algorithm achieves an approximation factor of 1.4 for the envy-ratio problem. In the next Theorem, we improve this result and show that we can achieve any constant factor arbitrarily close to 1 for the envy-ratio problem. Theorem 3.3. There is a PTAS for the envy-ratio problem when all players have the same utility for each good. Furthermore, when the number of players is constant, there is an FPTAS.

Expanded PaperInterface specification

∀ {SourceAgent : Type u_3} {load : SourceAgent → ℝ} {optimalRatio : ℝ}
 (C : LMMS04FairDivision.Theorem32.Graham14SchedulingApproximationCertificate load optimalRatio),
 LMMS04FairDivision.Theorem32.IdenticalUtilitiesEnvyRatioBound load
 (LMMS04FairDivision.Theorem32.grahamApproximationFactor * optimalRatio)

Lean proof endpoint (built separately)

LMMS04FairDivision.PaperInterface.theorem3_2_graham_certificate_to_envy_ratio_bound

Recorded source-to-Spec screening Verdict: not recorded

Reason: not recorded

Reviewer check: ☐ Matches source input

If left unchecked, explain the discrepancy or uncertainty below.

Reviewer annotation

Review row 14

Source locator: cited publication, lines 356-365

Verbatim source input

player whose current bundle has the least value. In [5] it was proved (in the context of scheduling) that the approximation factor of Graham's algorithm is 1.4 for the ratio problem. Theorem 3.2. [5] Graham's algorithm achieves an approximation factor of 1.4 for the envy-ratio problem. In the next Theorem, we improve this result and show that we can achieve any constant factor arbitrarily close to 1 for the envy-ratio problem. Theorem 3.3. There is a PTAS for the envy-ratio problem when all players have the same utility for each good. Furthermore, when the number of players is constant, there is an FPTAS.

Expanded PaperInterface specification

`LMS04FairDivision.Theorem32.grahamApproximationFactor = 7 / 5`

Lean proof endpoint (built separately)

`LMS04FairDivision.PaperInterface.theorem3_2_graham_factor_eq_seven_fifths`

Recorded source-to-Spec screening Verdict: not recorded

Reason: not recorded

Reviewer check: ☐ Matches source input

If left unchecked, explain the discrepancy or uncertainty below.

Reviewer annotation

--

Review row 15

Source locator: cited publication, lines 374-392

Verbatim source input

Proof. Before going into the details of the proof we give a brief outline of the technique. Our algorithm is similar to [2] and [20]. However our objective function does not fit in their framework. The algorithm is as follows: Given our original instance, we round the utility of each good to obtain a coarsest instance in which there is a constant number of distinct utilities (i.e., a constant number of different types of goods). We then show that in the new instance, we can find an optimal solution by searching for every player among a constant number of distinct assignments of goods. The constant will be exponential in the approximation parameter $1/[U+000F \text{ control character}]$. This observation enables us to compute the optimal solution in the rounded instance by solving a series of integer programs with a constant number of variables using Lenstra's algorithm [12]. After finding an optimal allocation in the rounded instance, we will convert it into an allocation for the original instance. In the whole process, there are 2 sources of error: computing the rounded instance from the original one and transforming the optimal allocation of the rounded instance to an allocation of the original instance. We are able to bound the error by $1 + [U+000F \text{ control character}]$.

Expanded PaperInterface specification

```
∀ {SourceAgent : Type u_3} {SourceItem : Type u_4} {Alloc : Type u_5} [inst : Fintype SourceAgent]
[inst_1 : Nonempty SourceAgent] [DecidableEq SourceItem]
{M : LMMS04FairDivision.Theorem33.IdenticalUtilitiesModel SourceAgent SourceItem Alloc} {L LR epsilon : ℝ}
{lambda : ℕ} {optimal : Alloc},
0 < epsilon →
epsilon ≤ 1 →
  ∀ (cert : LMMS04FairDivision.Theorem33.RoundedInstanceSearchCertificate M L epsilon lambda optimal)
    (highGoods lowGoods : Finset SourceItem) (v : SourceItem → ℝ),
    1 < lambda →
      0 < L →
        (∀ g ∈ highGoods, L / ↑lambda < v g ∧ v g < L) →
        (∀ g ∈ lowGoods, 0 ≤ v g ∧ v g < L / ↑lambda) →
        (∀ g ∈ highGoods, v g + L / ↑lambda ^ 2 ≤ L) →
        (∀ (index0f : SourceItem → LMMS04FairDivision.Theorem33.RoundedValueIndex lambda)
          (combinedSupply : LMMS04FairDivision.Theorem33.RoundedValueIndex lambda → ℕ),
          (∀ g ∈ highGoods,
            v g ≤ LMMS04FairDivision.Theorem33.roundedValue L lambda (index0f g) ∧
            LMMS04FairDivision.Theorem33.roundedValue L lambda (index0f g) < v g + L / ↑lambda ^ 2) →
          (∀ (k : LMMS04FairDivision.Theorem33.RoundedValueIndex lambda),
            combinedSupply k =
              LMMS04FairDivision.paper_lmms_theorem_3_3_roundedGoodsSupply0fGoods index0f highGoods k +
              LMMS04FairDivision.Theorem33.lowGoodsAggregatedSupply L lambda (lowGoods.sum v) k) →
          ∑ k, ↑(combinedSupply k) * LMMS04FairDivision.Theorem33.roundedValue L lambda k =
            ↑(Fintype.card SourceAgent) * LR) →
        (∀ (index0f : SourceItem → LMMS04FairDivision.Theorem33.RoundedValueIndex lambda)
          (combinedSupply : LMMS04FairDivision.Theorem33.RoundedValueIndex lambda → ℕ),
          (∀ g ∈ highGoods,
            v g ≤ LMMS04FairDivision.Theorem33.roundedValue L lambda (index0f g) ∧
            LMMS04FairDivision.Theorem33.roundedValue L lambda (index0f g) < v g + L / ↑lambda ^ 2) →
          (∀ (k : LMMS04FairDivision.Theorem33.RoundedValueIndex lambda),
            combinedSupply k =
              LMMS04FairDivision.paper_lmms_theorem_3_3_roundedGoodsSupply0fGoods index0f highGoods
                k +
              LMMS04FairDivision.Theorem33.lowGoodsAggregatedSupply L lambda (lowGoods.sum v) k) →
          combinedSupply (LMMS04FairDivision.paper_lmms_theorem_3_3_topRoundedValueIndex lambda) = 0 →
          ∑ k, ↑(combinedSupply k) * LMMS04FairDivision.Theorem33.roundedValue L lambda k =
            ↑(Fintype.card SourceAgent) * LR) →
        ∃ p,
          Nonempty
            (LMMS04FairDivision.Theorem33.RoundedConcreteIPCertificateWithCap L LR lambda
              (LMMS04FairDivision.paper_lmms_theorem_3_3_capCountForAverage L
                (L +
                  (↑(Fintype.card SourceAgent) + 1) * (L / ↑lambda) /
                  ↑(Fintype.card SourceAgent))
                lambda)
              SourceAgent p combinedSupply)) →
        ∃ maxCount ≤ 2 * lambda + 4,
          (LMMS04FairDivision.Theorem33.roundedAdmissibleTypeSetWithCap L LR lambda maxCount).card ≤
            (2 * lambda + 5) ^ Fintype.card (LMMS04FairDivision.Theorem33.RoundedValueIndex lambda) ∧
          (LMMS04FairDivision.Theorem33.roundedAdmissibleValuePairSetWithCap L LR lambda
            maxCount).card ≤
            ((2 * lambda + 5) ^
              Fintype.card (LMMS04FairDivision.Theorem33.RoundedValueIndex lambda)) ^
            2 ∧
          ∃ index0f combinedSupply p ip search,
            LMMS04FairDivision.Theorem33.allocationLoadRatio M cert.output ≤
              LMMS04FairDivision.Theorem33.allocationLoadRatio M optimal * (1 + epsilon) ∧
            LMMS04FairDivision.Theorem33.roundedValuePairRatio search.chosenPair ≤
              LMMS04FairDivision.Theorem33.roundedValuePairRatio p ∧
            p ∈
              LMMS04FairDivision.Theorem33.roundedAdmissibleValuePairSetWithCap L LR lambda
                maxCount ∧
            (LMMS04FairDivision.Theorem33.roundedTypesInValueWindowWithCap L LR lambda maxCount
              p.1 p.2).card ≤
              (2 * lambda + 5) ^
```

```

Fintype.card (LMMS04FairDivision.Theorem33.RoundedValueIndex lambda) ∧
(∀
  t ∉
    LMMS04FairDivision.Theorem33.roundedTypesInValueWindowWithCap L LR lambda
    maxCount p.1 p.2,
  ip.typeMultiplicity t = 0) ∧
(LMMS04FairDivision.Theorem33.roundedTypesInValueWindowWithCap L LR lambda
  maxCount p.1 p.2).sum
  ip.typeMultiplicity =
  Fintype.card SourceAgent ∧
(∀ (k : LMMS04FairDivision.Theorem33.RoundedValueIndex lambda),
  ∑
    t ∈
      LMMS04FairDivision.Theorem33.roundedTypesInValueWindowWithCap L LR
      lambda maxCount p.1 p.2,
      ip.typeMultiplicity t * t.count k =
      combinedSupply k) ∧
  search.chosenPair ∈
    LMMS04FairDivision.Theorem33.roundedAdmissibleValuePairSetWithCap L LR
    lambda maxCount ∧
  (LMMS04FairDivision.Theorem33.roundedTypesInValueWindowWithCap L LR lambda
    maxCount search.chosenPair.1 search.chosenPair.2).card ≤
    (2 * lambda + 5) ^
    Fintype.card
    (LMMS04FairDivision.Theorem33.RoundedValueIndex lambda) ∧
  (∀
    t ∉
      LMMS04FairDivision.Theorem33.roundedTypesInValueWindowWithCap L LR
      lambda maxCount search.chosenPair.1 search.chosenPair.2,
      search.chosenIP.typeMultiplicity t = 0) ∧
  (LMMS04FairDivision.Theorem33.roundedTypesInValueWindowWithCap L LR
    lambda maxCount search.chosenPair.1 search.chosenPair.2).sum
    search.chosenIP.typeMultiplicity =
    Fintype.card SourceAgent ∧
  (∀ (k : LMMS04FairDivision.Theorem33.RoundedValueIndex lambda),
    ∑
      t ∈
        LMMS04FairDivision.Theorem33.roundedTypesInValueWindowWithCap
        L LR lambda maxCount search.chosenPair.1
        search.chosenPair.2,
        search.chosenIP.typeMultiplicity t * t.count k =
        combinedSupply k) ∧
    (∀ g ∈ highGoods,
      v g ≤
        LMMS04FairDivision.Theorem33.roundedValue L lambda
        (indexOf g) ∧
        LMMS04FairDivision.Theorem33.roundedValue L lambda
        (indexOf g) <
        v g + L / ↑lambda ^ 2) ∧
    (∀ (k : LMMS04FairDivision.Theorem33.RoundedValueIndex lambda),
      combinedSupply k =
        LMMS04FairDivision.paper_lmms_theorem_3_3_roundedGoodsSupplyOfGoods
        indexOf highGoods k +
        LMMS04FairDivision.Theorem33.lowGoodsAggregatedSupply L
        lambda (lowGoods.sum v) k) ∧
    combinedSupply
      (LMMS04FairDivision.paper_lmms_theorem_3_3_topRoundedValueIndex
        lambda) =
    0
  )

```

Lean proof endpoint (built separately)

LMMS04FairDivision.PaperInterface.theorem3_3_solver_auto_cap_full_ip_summary_with_ratio_guarantee

Recorded source-to-Spec screening Verdict: not recorded

Reason: not recorded

Reviewer check: ☐ Matches source input

If left unchecked, explain the discrepancy or uncertainty below.

Reviewer annotation

Review row 16

Source locator: cited publication, lines 681-719

Verbatim source input

Since AR is an optimal solution in I R the ratio in A0 will be at least as big as in AR . Hence by combining the above equations we finally have:

$$\begin{aligned} & v(A_n) \\ & v(A_1) \\ & \leq \\ & \frac{\lambda+1}{2} \\ & v(A \cdot n) \cdot 1 + \lambda_2 \\ & \left(\frac{\lambda}{2} \right) \left(\frac{\lambda}{2} \right) \\ & * \\ & 4 \\ & v(A_1) \cdot \lambda+1 - \lambda \\ & 1 - \lambda \\ & \leq \\ & (\lambda + 1)(\lambda + 2)(\lambda + 3) \\ & OP \ T \\ & (\lambda - 2)(\lambda_2 - 4\lambda - 4) \end{aligned}$$

be envious). But then player 2 will receive a and at most 25 eggs so she will be envious, a contradiction. Therefore in A player 1 receives a and T eggs and player 2 receives b and k - T eggs. It is also easy to see that $25 \leq T \leq 75$.

Case 1: $T < 74$

In this case player 1 can increase her utility by lying and declaring that good a has less value for her. It is possible for her to lie in such a way to force the mechanism to give her the good a and at least $T + 1$ eggs (assuming that 2 does not change her declaration). She can declare that her valuation function is: $v_1(a) = 0.45 - \delta$, $v_1(b) = 0.35 + \delta$, $v_1(\text{egg}) = 0.2/k$ where δ is such that:

Thus, if we set $\lambda = 56/[U+000F \text{ control character}]$, it is easy to see that the factor will be at most $1 + [U+000F \text{ control character}]$.

Expanded PaperInterface specification

```
∀ {SourceAgent : Type u_3} {SourceItem : Type u_4} {Alloc : Type u_5} [inst : Fintype SourceAgent]
[inst_1 : Nonempty SourceAgent] [DecidableEq SourceAgent] [DecidableEq SourceItem]
{M : LMMS04FairDivision.Theorem33.IdenticalUtilitiesModel SourceAgent SourceItem Alloc} {L epsilon : ℝ} {lambda : ℕ}
{optimal : Alloc},
0 < epsilon →
epsilon ≤ 1 →
∀ (cert : LMMS04FairDivision.Theorem33.RoundedInstanceSearchCertificate M L epsilon lambda optimal)
(highGoods lowGoods : Finset SourceItem) (v : SourceItem → ℝ)
(indexOf : SourceItem → LMMS04FairDivision.Theorem33.RoundedValueIndex lambda)
(combinedSupply : LMMS04FairDivision.Theorem33.RoundedValueIndex lambda → ℕ),
1 < lambda →
0 < L →
(∀ g ∈ highGoods,
  v g ≤ LMMS04FairDivision.Theorem33.roundedValue L lambda (indexOf g) ∧
  LMMS04FairDivision.Theorem33.roundedValue L lambda (indexOf g) < v g + L / ↑lambda ^ 2) →
(∀ (k : LMMS04FairDivision.Theorem33.RoundedValueIndex lambda),
  combinedSupply k =
    LMMS04FairDivision.paper_lmms_theorem_3_3_roundedGoodsSupplyOfGoods indexOf highGoods k +
    LMMS04FairDivision.Theorem33.lowGoodsAggregatedSupply L lambda (lowGoods.sum v) k) →
(∀ g ∈ highGoods, v g + L / ↑lambda ^ 2 ≤ L) →
∑ k, ↑(combinedSupply k) * LMMS04FairDivision.Theorem33.roundedValue L lambda k =
  ↑(Fintype.card SourceAgent) * L →
  LMMS04FairDivision.Theorem33.allocationLoadRatio M cert.output ≤
  LMMS04FairDivision.Theorem33.allocationLoadRatio M optimal * (1 + epsilon) ∧
  combinedSupply (LMMS04FairDivision.paper_lmms_theorem_3_3_topRoundedValueIndex lambda) = 0 ∧
  ∃ p typeOf,
    p ∈ LMMS04FairDivision.Theorem33.roundedAdmissibleValuePairSet L lambda ∧
    (∀ (i : SourceAgent),
      typeOf i ∈ LMMS04FairDivision.Theorem33.roundedTypesInValueWindow L lambda p.1 p.2) ∧
    (∀ (k : LMMS04FairDivision.Theorem33.RoundedValueIndex lambda),
      ∑ i, (typeOf i).count k = combinedSupply k) ∧
    ∃ search,
      LMMS04FairDivision.Theorem33.roundedValuePairRatio search.chosenPair ≤
      LMMS04FairDivision.Theorem33.roundedValuePairRatio p
```

Lean proof endpoint (built separately)

LMMS04FairDivision.PaperInterface.theorem3_3_claim34_fixed_rounding_ratio_endpoint

Recorded source-to-Spec screening Verdict: not recorded

Reason: not recorded

Reviewer check: ☐ Matches source input

If left unchecked, explain the discrepancy or uncertainty below.

Reviewer annotation

--

Review row 17

Source locator: cited publication, lines 494-550

Verbatim source input

Claim 3.4. If $v(i) < L$ for every good i , then there exists an optimal allocation $A = (A_1, \dots, A_n)$ such that $2L < v(A_i) < 2L$.

The proof is by showing that in a given optimal solution, it is possible to reallocate the goods so that the envy-ratio does not increase and the conditions of the claim are satisfied. We will now describe how to round the values of the goods and obtain an instance in which there is only a constant number of different types of goods (i.e. a constant number of distinct values for the goods). The construction is the same as in [2] and we repeat it here for the sake of completeness. We will denote the rounded instance by $I_R(\lambda)$, where λ is a positive constant and will be determined later (λ will be $O(1/\epsilon)$ control character)). We will often omit λ in the notation. We first round the value of every good with relatively high value. In particular, for every good i with $v(i) > L/\lambda$, we round $v(i)$ to the next integer multiple of L/λ^2 . Roughly this means that we round up the first least significant digits of $v(i)$. We cannot afford to do the same for goods with small value since the error introduced by this process might be very big. Instead, let S be the sum of the values of the goods with value less than L/λ . We round S to the next integer multiple of L/λ , say S_R . Instance $I_R(\lambda)$ will have $S_R \lambda/L$ new goods with value L/λ . This completes the construction. Note that in $I_R(\lambda)$ all values are of the form kL/λ^2 , where $\lambda \leq k \leq \lambda^2$. Hence we have only a constant number of distinct values.

Let M_R be the set of goods in the newP instance and $v_R(i)$

be the value of each good. Let $L = \sum_{i \in M_R} v_R(i)$. It is easy to see that $L \leq LR$ and that all values in $I_R(\lambda)$ are at most LR . Hence by Claim 3.4 there is an optimal solution

$A = (A_1, \dots, A_n)$ such that $2L < v(A_i) < 2L$ for every i . In the algorithm below we will search for such a solution. We represent a player's bundle by a vector $t = (t_1, \dots, t_{\lambda^2})$, where t_k is the number of goods with value kL/λ^2 assigned to her. We will then say that the player is of type t . The utility derived from t is $v(t) = \sum_k t_k kL/\lambda^2$. Let U be the set of all possible types t , with $2L < v(t) < 2LR$. It is easy to see that $|U|$ is bounded by a constant which is exponential in λ . Hence for a player of type $t \in U$, there is only a constant number of distinct values for her utility. Let $V(U)$ be the set of these values, i.e. $V(U) = \{u : v(t) = u \text{ for some } t \in U\}$. We can now show how to find the optimal envy-ratio in $I_R(\lambda)$. For all pairs of values $u_1, u_2 \in V(U)$, we will solve the following decision problem: Is there an allocation in which the utility of every player is between u_1 and u_2 ? Since $|V(U)|$ is constant, after solving the above problem for all u_1, u_2 we can output the allocation corresponding to u^*_1, u^*_2 for which the minimum ratio u^*_2/u^*_1 is attained.

Expanded PaperInterface specification

```
∀ {SourceAgent : Type u_3} {SourceItem : Type u_4} {Alloc : Type u_5} [inst : Fintype SourceAgent]
[inst_1 : Nonempty SourceAgent] [DecidableEq SourceAgent]
{M : LMMS04FairDivision.Theorem33.IdenticalUtilitiesModel SourceAgent SourceItem Alloc} {L LR epsilon : ℝ}
{lambda maxCount : ℕ} {optimal : Alloc},
0 < epsilon →
epsilon ≤ 1 →
  ∀ (cert : LMMS04FairDivision.Theorem33.RoundedInstanceSearchCertificate M L epsilon lambda optimal)
  (supply : LMMS04FairDivision.Theorem33.RoundedValueIndex lambda → ℕ),
  ∑ k, ↑(supply k) * LMMS04FairDivision.Theorem33.roundedValue L lambda k = ↑(Fintype.card SourceAgent) * LR →
  L ≤ LR →
    supply (LMMS04FairDivision.paper_lmms_theorem_3_3_topRoundedValueIndex lambda) = 0 →
    2 * LR ≤ (↑maxCount + 1) * (L / ↑lambda) →
    0 < lambda →
    0 < L →
    0 < LR →
      LMMS04FairDivision.Theorem33.allocationLoadRatio M cert.output ≤
      LMMS04FairDivision.Theorem33.allocationLoadRatio M optimal * (1 + epsilon) ∧
      ∃ p typeOf,
      p ∈ LMMS04FairDivision.Theorem33.roundedAdmissibleValuePairSetWithCap L LR lambda maxCount ∧
      (∀ (i : SourceAgent),
        typeOf i ∈
        LMMS04FairDivision.Theorem33.roundedTypesInValueWindowWithCap L LR lambda maxCount p.1
        p.2) ∧
      (∀ (k : LMMS04FairDivision.Theorem33.RoundedValueIndex lambda),
        ∑ i, (typeOf i).count k = supply k) ∧
      ∃ search,
      LMMS04FairDivision.Theorem33.roundedValuePairRatio search.chosenPair ≤
      LMMS04FairDivision.Theorem33.roundedValuePairRatio p
```

Lean proof endpoint (built separately)

LMS04FairDivision.PaperInterface.theorem3_3_claim34_capped_weighted_supply_ratio_endpoint

Recorded source-to-Spec screening Verdict: not recorded

Reason: not recorded

Reviewer check: ☐ Matches source input

If left unchecked, explain the discrepancy or uncertainty below.

Reviewer annotation

Review row 18

Source locator: cited publication, lines 491-498

Verbatim source input

We omit the details for handling goods with $v(i) \geq L$ and from now on we will assume that $v(i) < L$ for every i . We have the following fact:

Claim 3.4. If $v(i) < L$ for every good i , then there exists an optimal allocation $A = (A_1, \dots, A_n)$ such that $12L < v(A_i) < 2L$.

The proof is by showing that in a given optimal solution, it is possible to reallocate the goods so that the envy-ratio does not increase and the conditions of the claim are satisfied.

Expanded PaperInterface specification

```
∀ {SourceAgent : Type u_3} {SourceItem : Type u_4} [inst : Fintype SourceAgent] [Fintype SourceItem]
[inst_2 : Nonempty SourceAgent] [DecidableEq SourceAgent] [inst_4 : DecidableEq SourceItem]
{goods : Finset SourceItem} {v : SourceItem → ℝ} {L : ℝ}
(A₀ : EconCSLib.FairDivision.Allocation SourceAgent SourceItem),
LMMS04FairDivision.Theorem34.IsOptimalRatio
  (fun A =>
    LMMS04FairDivision.Theorem34.loadRatio (LMMS04FairDivision.Theorem34.minCommonLoad v A)
      (LMMS04FairDivision.Theorem34.maxCommonLoad v A))
  A₀ →
  (∀ (A : EconCSLib.FairDivision.Allocation SourceAgent SourceItem),
    LMMS04FairDivision.Theorem34.IsOptimalRatio
      (fun C =>
        LMMS04FairDivision.Theorem34.loadRatio (LMMS04FairDivision.Theorem34.minCommonLoad v C)
          (LMMS04FairDivision.Theorem34.maxCommonLoad v C))
      A →
      EconCSLib.FairDivision.IsAllocationOf A goods) →
  0 < L →
  LMMS04FairDivision.Theorem34.commonLoad v goods = ↑(Fintype.card SourceAgent) * L →
  (∀ (A : EconCSLib.FairDivision.Allocation SourceAgent SourceItem),
    LMMS04FairDivision.Theorem34.IsOptimalRatio
      (fun C =>
        LMMS04FairDivision.Theorem34.loadRatio (LMMS04FairDivision.Theorem34.minCommonLoad v C)
          (LMMS04FairDivision.Theorem34.maxCommonLoad v C))
      A →
      (¬∀ (i : SourceAgent),
        LMMS04FairDivision.Theorem34.boundedAroundAverage L
          (LMMS04FairDivision.Theorem34.commonLoad v (A i))) →
      ∀ (i : SourceAgent), ∃ g, g ∈ A i) →
  (∀ g ∈ goods, 0 < v g) →
  (∀ g ∈ goods, v g < L) →
  ∃ B,
    EconCSLib.FairDivision.IsAllocationOf B goods ∧
    LMMS04FairDivision.Theorem34.Claim34Certificate v L
      (fun A =>
        LMMS04FairDivision.Theorem34.loadRatio (LMMS04FairDivision.Theorem34.minCommonLoad v A)
          (LMMS04FairDivision.Theorem34.maxCommonLoad v A))
      (fun A i => A i) B
```

Lean proof endpoint (built separately)

LMMS04FairDivision.PaperInterface.claim3_4_bounded_optimal_of_exact_allocations_with_nonempty_positive_goods

Recorded source-to-Spec screening Verdict: not recorded

Reason: not recorded

Reviewer check: ☐ Matches source input

If left unchecked, explain the discrepancy or uncertainty below.

Reviewer annotation

Review row 19

Source locator: cited publication, lines 491-498

Verbatim source input

We omit the details for handling goods with $v(i) \geq L$ and from now on we will assume that $v(i) < L$ for every i . We have the following fact:
Claim 3.4. If $v(i) < L$ for every good i , then there exists an optimal allocation $A = (A_1, \dots, A_n)$ such that $\sum L < v(A_i) < 2L$.
The proof is by showing that in a given optimal solution, it is possible to reallocate the goods so that the envy-ratio does not increase and the conditions of the claim are satisfied.

Expanded PaperInterface specification

```
∀ {SourceAgent : Type u_3} {SourceItem : Type u_4} {Alloc : Type u_5} [inst : Fintype SourceAgent] [Fintype Alloc]
[inst_2 : Nonempty SourceAgent] [inst_3 : DecidableEq SourceAgent] [inst_4 : DecidableEq SourceItem]
(M : LMMS04FairDivision.Theorem33.IdenticalUtilitiesModel SourceAgent SourceItem Alloc) {L : ℝ}
(moveAlloc : Alloc → SourceAgent → SourceItem → Alloc) (A₀ : Alloc),
LMMS04FairDivision.Theorem34.IsOptimalRatio (LMMS04FairDivision.Theorem33.allocationLoadRatio M) A₀ →
0 < L →
(∀ (A : Alloc),
  LMMS04FairDivision.Theorem34.IsOptimalRatio (LMMS04FairDivision.Theorem33.allocationLoadRatio M) A →
  ∑ i, LMMS04FairDivision.Theorem34.commonLoad M.commonValue (M.bundleOf A i) =
  ↑(Fintype.card SourceAgent) * L) →
(∀ (A : Alloc),
  LMMS04FairDivision.Theorem34.IsOptimalRatio (LMMS04FairDivision.Theorem33.allocationLoadRatio M) A →
  (¬∀ (i : SourceAgent),
    LMMS04FairDivision.Theorem34.boundedAroundAverage L
    (LMMS04FairDivision.Theorem34.commonLoad M.commonValue (M.bundleOf A i))) →
  0 < LMMS04FairDivision.Theorem33.minAllocationLoad M A) →
(∀ (A : Alloc) (i : SourceAgent), ∀ g ∈ M.bundleOf A i, 0 ≤ M.commonValue g) →
(∀ (A : Alloc) (i : SourceAgent), ∀ g ∈ M.bundleOf A i, M.commonValue g < L) →
(∀ (A : Alloc) {i j : SourceAgent} {g : SourceItem}, g ∈ M.bundleOf A i → g ∈ M.bundleOf A j → i = j) →
(∀ (A : Alloc) (source target : SourceAgent) (g : SourceItem),
  M.bundleOf (moveAlloc A source target g) =
  LMMS04FairDivision.Theorem34.moveBundle (M.bundleOf A) source target g) →
∃ B,
  LMMS04FairDivision.Theorem34.Claim34Certificate M.commonValue L
  (LMMS04FairDivision.Theorem33.allocationLoadRatio M) M.bundleOf B
```

Lean proof endpoint (built separately)

LMMS04FairDivision.PaperInterface.claim3_4_identical_utilities_bounded_optimum_bound

Recorded source-to-Spec screening Verdict: not recorded

Reason: not recorded

Reviewer check: ☐ Matches source input

If left unchecked, explain the discrepancy or uncertainty below.

Reviewer annotation

Review row 20

Source locator: cited publication, lines 608-719

Verbatim source input

We are now ready to prove our Theorem. Our algorithm will be: Given instance I , compute the instance I^R , find R an optimal allocation $AR = (AR_1, \dots, AR_n)$ for I^R , and then convert A to an allocation $A = (A_1, \dots, A_n)$ for I using Lemma 3.5. Output A . Suppose without loss of generality that $v(AR_1) \leq \dots \leq v(AR_n)$ and $v(A_1) \leq \dots \leq v(A_n)$. Let $A = (A_1, \dots, A_n)$ be an optimal solution to I satisfying the conditions of Claim 3.4 and assume $v(A_1) \leq \dots \leq v(A_n)$. We want to show:

$$\frac{v(A_n)}{v(A_1)} \leq (1 + \frac{1}{\lambda}) \frac{v(AR_n)}{v(AR_1)}$$

By Lemma 3.5 we know that:

$$\frac{v(A_n)}{v(A_1)} \leq \frac{v(AR_n)}{v(AR_1)} + \frac{1}{\lambda} \left(\frac{v(AR_n)}{v(AR_1)} - 1 \right)$$

Similar calculations yield: $\frac{v(A_1)}{v(A_n)} \geq \frac{v(AR_1)}{v(AR_n)} - \frac{1}{\lambda} \left(\frac{v(AR_1)}{v(AR_n)} - 1 \right)$. Therefore:

$$\frac{v(A_n)}{v(A_1)} \leq \frac{v(AR_n)}{v(AR_1)} + \frac{1}{\lambda} \left(\frac{v(AR_n)}{v(AR_1)} - 1 \right) + \frac{1}{\lambda} \left(1 - \frac{v(AR_1)}{v(AR_n)} \right)$$

We need to relate the optimal solution in I^R with the optimal solution in I . By using the first part of Lemma 3.5 and by performing similar calculations we have that there exists an allocation $A_0 = (A_{01}, \dots, A_{0n})$ in I^R such that:

$$\frac{v(A_{0n})}{v(A_{01})} \leq \frac{v(AR_n)}{v(AR_1)} + \frac{1}{\lambda} \left(\frac{v(AR_n)}{v(AR_1)} - 1 \right)$$

[form-feed in source extraction]

Since AR is an optimal solution in I^R the ratio in A_0 will be at least as big as in AR . Hence by combining the above equations we finally have:

$$\frac{v(A_n)}{v(A_1)} \leq \frac{v(AR_n)}{v(AR_1)} + \frac{1}{\lambda} \left(\frac{v(AR_n)}{v(AR_1)} - 1 \right) + \frac{1}{\lambda} \left(1 - \frac{v(AR_1)}{v(AR_n)} \right)$$

$$v(A1) \lambda + 1 - \lambda$$

$$1 - \lambda$$

$$\leq$$

$$(\lambda + 1)(\lambda + 2)(\lambda + 3)$$

$$OP \ T$$

$$(\lambda - 2)(\lambda^2 - 4\lambda - 4)$$

be envious). But then player 2 will receive a and at most 25 eggs so she will be envious, a contradiction. Therefore in A player 1 receives a and T eggs and player 2 receives b and k - T eggs. It is also easy to see that $25 \leq T \leq 75$.

Case 1: $T < 74$

In this case player 1 can increase her utility by lying and declaring that good a has less value for her. It is possible for her to lie in such a way to force the mechanism to give her the good a and at least $T + 1$ eggs (assuming that 2 does not change her declaration). She can declare that her valuation function is: $v_1(a) = 0.45 - \delta$, $v_1(b) = 0.35 + \delta$, $v_1(\text{egg}) = 0.2/k$ where δ is such that:

Thus, if we set $\lambda = 56/[U+000F \text{ control character}]$, it is easy to see that the factor will be at most $1 + [U+000F \text{ control character}]$.

Expanded PaperInterface specification

```

∀ {SourceAgent : Type u_3} [inst : Fintype SourceAgent] [inst_1 : Nonempty SourceAgent] {epsilon L : ℝ}
{optLoad roundedLoad outLoad : SourceAgent → ℝ},
0 < epsilon →
epsilon ≤ 1 →
0 < Finset.univ.inf' ... outLoad →
0 < Finset.univ.inf' ... roundedLoad →
0 < Finset.univ.inf' ... optLoad →
0 ≤ Finset.univ.sup' ... optLoad →
0 ≤ Finset.univ.sup' ... roundedLoad →
L ≤ 2 * Finset.univ.inf' ... roundedLoad →
L ≤ 2 * Finset.univ.sup' ... roundedLoad →
L ≤ 2 * Finset.univ.inf' ... optLoad →
L ≤ 2 * Finset.univ.sup' ... optLoad →
(∀ (i : SourceAgent), outLoad i ≤ roundedLoad i + L / (56 / epsilon)) →
(∀ (i : SourceAgent),
56 / epsilon / (56 / epsilon + 1) * roundedLoad i - 2 / (56 / epsilon) * L ≤ outLoad i) →
(∀ (i : SourceAgent),
roundedLoad i ≤ (56 / epsilon + 1) / (56 / epsilon) * optLoad i + L / (56 / epsilon)) →
(∀ (i : SourceAgent), optLoad i - L / (56 / epsilon) ≤ roundedLoad i) →
LMMS04FairDivision.Theorem35.ratio (Finset.univ.inf' ... outLoad)
(Finset.univ.sup' ... outLoad) ≤
LMMS04FairDivision.Theorem35.ratio (Finset.univ.inf' ... optLoad)
(Finset.univ.sup' ... optLoad) *
(1 + epsilon)

```

Lean proof endpoint (built separately)

LMMS04FairDivision.PaperInterface.theorem3_3_ratio_transfer_certificate_epsilon_of_agentwise_additive_loads

Recorded source-to-Spec screening Verdict: not recorded

Reason: not recorded

Reviewer check: ☐ Matches source input

If left unchecked, explain the discrepancy or uncertainty below.

Reviewer annotation

Review row 21

Source locator: cited publication, lines 582-607

Verbatim source input

Lemma 3.5. Let $A = (A_1, \dots, A_n)$ be an allocation in I , where $12L < v(A_i) < 2L$. Then there exists an allocation $B = (B_1, \dots, B_n)$ in the rounded instance, I_R , such that: $v(A_i) -$

λ
 $\lambda+1$
 λ
 $L \leq v(B_i) \leq$
 $v(A_i) + L$
 λ
 λ
 λ

Also if $B = (B_1, \dots, B_n)$ is an allocation in I_R such that

λ
 $L < v(B_i) < 2LR$, then there exists an allocation $A =$
 λ
 $(A_1, \dots, A_n)$, in I such that:
 λ
 λ
 λ
 $v(B_i) - L \leq v(A_i) \leq v(B_i) + L$
 $\lambda+1$
 λ
 λ

Expanded PaperInterface specification

```
∀ {epsilon L optMin optMax roundedMin roundedMax outMin outMax : ℝ},
  0 < epsilon →
    epsilon ≤ 1 →
      0 < outMin →
        0 < roundedMin →
          0 < optMin →
            0 ≤ optMax →
              0 ≤ roundedMax →
                L ≤ 2 * roundedMin →
                  L ≤ 2 * roundedMax →
                    L ≤ 2 * optMin →
                      L ≤ 2 * optMax →
                        outMax ≤ roundedMax + L / (56 / epsilon) →
                          56 / epsilon / (56 / epsilon + 1) * roundedMin - 2 / (56 / epsilon) * L ≤ outMin →
                            roundedMax ≤ (56 / epsilon + 1) / (56 / epsilon) * optMax + L / (56 / epsilon) →
                              optMin - L / (56 / epsilon) ≤ roundedMin →
                                LMS04FairDivision.Theorem35.ratio outMin outMax ≤
                                  LMS04FairDivision.Theorem35.ratio optMin optMax * (1 + epsilon)
```

Lean proof endpoint (built separately)

LMS04FairDivision.PaperInterface.lemma3_5_additive_transfer_epsilon_bound

Recorded source-to-Spec screening Verdict: not recorded

Reason: not recorded

Reviewer check: ☐ Matches source input

If left unchecked, explain the discrepancy or uncertainty below.

Reviewer annotation

Review row 22

Source locator: cited publication, lines 740-758

Verbatim source input

TRUTHFULNESS

So far we have assumed that we can obtain the actual utilities of the players for the goods. However, in many situations players' utilities are private information and they might lie about their valuations in order to obtain a better bundle. We would like to investigate the question of whether we can have mechanisms that elicit truthful utilities from the players and also produce allocations with minimum or bounded envy. A mechanism is truthful if for every player, her profit is maximized by declaring her true utility i.e. being truthful is a dominant strategy. In the last few years, truthful mechanisms have been the subject of research especially in the context of auctions. However, unlike our problem, auction mechanisms are allowed to compensate the players with money to keep them truthful. We will present a simple argument to prove that any mechanism that computes a minimum-envy allocation can not be truthful even in the special case where the utility functions are additive.

In the rest of the section, we present a simple truthful

Expanded PaperInterface specification

$\forall$ (Agent : Type u_3) (Item : Type u_4) (M : LMMS04FairDivision.PaperInterface.directMechanism Agent Item), M = M

Lean proof endpoint (built separately)

LMMS04FairDivision.PaperInterface.directMechanism_fields

Recorded source-to-Spec screening Verdict: not recorded

Reason: not recorded

Reviewer check: ☐ Matches source input

If left unchecked, explain the discrepancy or uncertainty below.

Reviewer annotation

Review row 23

Source locator: cited publication, lines 758-776

Verbatim source input

In the rest of the section, we present a simple truthful mechanism which allocates the goods to the players uniformly at random. We assume that the sum of the utilities of each player over all goods is one. We will show that with high probability the maximum envy of the resulting allocation is no more than $O(\alpha^{1/2} + \frac{1}{n})$.
Theorem 4.2. Suppose that $u_i \leq \alpha \forall p \in N, j \in M$. Then for every $\epsilon > 0$, and for large enough n , there exists a truthful algorithm such that with high probability the allocation output by the algorithm has maximum envy at most $O(\alpha^{1/2} + \frac{1}{n})$.

Theorem 4.1. Any mechanism that returns an allocation with minimum possible envy cannot be truthful. The same is true for any mechanism that returns an envy-free allocation whenever there exists one.

Proof. The proof is based on the probabilistic method. Allocate each good independently to player p with probability $1/n$. Clearly this is a truthful mechanism. We will show

Expanded PaperInterface specification

$\forall$ (Agent : Type u_3) (Item : Type u_4) (M : LMS04FairDivision.PaperInterface.randomizedDirectMechanism Agent Item),
M = M

Lean proof endpoint (built separately)

LMS04FairDivision.PaperInterface.randomizedDirectMechanism_fields

Recorded source-to-Spec screening Verdict: not recorded

Reason: not recorded

Reviewer check: ☐ Matches source input

If left unchecked, explain the discrepancy or uncertainty below.

Reviewer annotation

Review row 24

Source locator: cited publication, lines 740-749

Verbatim source input

TRUTHFULNESS

So far we have assumed that we can obtain the actual utilities of the players for the goods. However, in many situations players' utilities are private information and they might lie about their valuations in order to obtain a better bundle. We would like to investigate the question of whether we can have mechanisms that elicit truthful utilities from the players and also produce allocations with minimum or bounded envy. A mechanism is truthful if for every player, her profit is maximized by declaring her true utility i.e. being truthful is a dominant strategy.

Expanded PaperInterface specification

```
∀ {Agent : Type u_1} {Item : Type u_2} [inst : DecidableEq Agent]
  (M : LMS04FairDivision.PaperInterface.directMechanism Agent Item),
  LMS04FairDivision.PaperInterface.truthful M = LMS04FairDivision.paper_fair_division_truthful M
```

Lean proof endpoint (built separately)

LMS04FairDivision.PaperInterface.truthful

Recorded source-to-Spec screening Verdict: not recorded

Reason: not recorded

Reviewer check: ☐ Matches source input

If left unchecked, explain the discrepancy or uncertainty below.

Reviewer annotation

Review row 25

Source locator: cited publication, lines 758-776

Verbatim source input

In the rest of the section, we present a simple truthful mechanism which allocates the goods to the players uniformly at random. We assume that the sum of the utilities of each player over all goods is one. We will show that with high probability the maximum envy of the resulting allocation is no more than $O(\alpha^{1/2} + \alpha \sum_{i=1}^n \sum_{j=1}^m u_{ij})$.
Theorem 4.2. Suppose that $u_{ij} \leq \alpha \forall p \in N, j \in M$. Then for every $\epsilon > 0$, and for large enough n , there exists a truthful algorithm such that with high probability the allocation $\sqrt{\epsilon}$ output by the algorithm has maximum envy at most $O(\alpha^{1/2} + \alpha \sum_{i=1}^n \sum_{j=1}^m u_{ij})$.

Theorem 4.1. Any mechanism that returns an allocation with minimum possible envy cannot be truthful. The same is true for any mechanism that returns an envy-free allocation whenever there exists one.

Proof. The proof is based on the probabilistic method. Allocate each good independently to player p with probability $1/n$. Clearly this is a truthful mechanism. We will show

Expanded PaperInterface specification

```
∀ {Agent : Type u_1} {Item : Type u_2} [inst : Fintype (EconCSLib.FairDivision.Allocation Agent Item)]
  [inst_1 : DecidableEq (EconCSLib.FairDivision.Allocation Agent Item)] [inst_2 : DecidableEq Agent]
  (M : LMMS04FairDivision.PaperInterface.randomizedDirectMechanism Agent Item),
  LMMS04FairDivision.PaperInterface.randomizedTruthful M = LMMS04FairDivision.paper_randomized_fair_division_truthful M
```

Lean proof endpoint (built separately)

LMMS04FairDivision.PaperInterface.randomizedTruthful

Recorded source-to-Spec screening Verdict: not recorded

Reason: not recorded

Reviewer check: ☐ Matches source input

If left unchecked, explain the discrepancy or uncertainty below.

Reviewer annotation

Review row 26

Source locator: cited publication, lines 795-814

Verbatim source input

Proof. Let M be a mechanism that outputs an allocation with minimum envy. Consider the following instance: We have two players, 1 and 2 and $k + 2$ goods. In particular we have good a , good b and k eggs. Let's say $k = 100$. The k eggs are playing the role of an almost divisible good of value 0.2 . Suppose the players have the following utilities for the goods:

$v_1(a) = 0.45$, $v_1(b) = 0.35$, $v_1(\text{egg}) = 0.2/k$,

$v_2(a) = 0.35$, $v_2(b) = 0.45$, $v_2(\text{egg}) = 0.2/k$

The specific instance admits an envy-free allocation: give to player 1 good a and 25 eggs and give the rest to player 2. Therefore in the allocation that M will output there will be no envy. Let A be the partition that M outputs for this instance. Note that in A each player receives exactly one of the goods a , b because if one player received both then the other player would envy her. Also we note that it is Player 1 who receives a . To see this, suppose on the contrary that player 1 receives b . Then in order for A to be envy-free player 1 should receive at least 75 eggs (otherwise the bundle S of player 1 is worth less than $1/2$ and she will

Expanded PaperInterface specification

```
LMMS04FairDivision.PaperInterface.theorem4_1_source_goods = Finset.univ ∧  
  Fintype.card LMMS04FairDivision.Theorem41.LMMS41Agent = 2 ∧  
  LMMS04FairDivision.PaperInterface.theorem4_1_source_goods.card = 10 ∧  
  LMMS04FairDivision.Theorem41.LMMS41EggItems.card = 8
```

Lean proof endpoint (built separately)

```
LMMS04FairDivision.PaperInterface.theorem4_1_source_goods_content
```

Recorded source-to-Spec screening Verdict: not recorded

Reason: not recorded

Reviewer check: ☐ Matches source input

If left unchecked, explain the discrepancy or uncertainty below.

Reviewer annotation

Review row 27

Source locator: cited publication, lines 795-803

Verbatim source input

Proof. Let M be a mechanism that outputs an allocation with minimum envy. Consider the following instance:
We have two players, 1 and 2 and $k + 2$ goods. In particular
we have good a , good b and k eggs. Let's say $k = 100$. The
 k eggs are playing the role of an almost divisible good of
value 0.2. Suppose the players have the following utilities
for the goods:
 $v_1(a) = 0.45$, $v_1(b) = 0.35$, $v_1(\text{egg}) = 0.2/k$,
 $v_2(a) = 0.35$, $v_2(b) = 0.45$, $v_2(\text{egg}) = 0.2/k$
The specific instance admits an envy-free allocation: give

Expanded PaperInterface specification

```
LMS04FairDivision.PaperInterface.theorem4_1_true_report =  
  LMS04FairDivision.Theorem41.lms41AdditiveReport LMS04FairDivision.Theorem41.lms41TrueWeight  $\wedge$   
   $\forall$  (agent : LMS04FairDivision.Theorem41.LMS41Agent) (item : LMS04FairDivision.Theorem41.LMS41Item),  
    LMS04FairDivision.Theorem41.lms41TrueWeight agent item =  
      if agent = LMS04FairDivision.Theorem41.LMS41Agent.player1 then  
        if item = LMS04FairDivision.Theorem41.LMS41Item.a then 9 / 20  
        else if item = LMS04FairDivision.Theorem41.LMS41Item.b then 7 / 20 else 1 / 40  
      else  
        if item = LMS04FairDivision.Theorem41.LMS41Item.a then 7 / 20  
        else if item = LMS04FairDivision.Theorem41.LMS41Item.b then 9 / 20 else 1 / 40
```

Lean proof endpoint (built separately)

```
LMS04FairDivision.PaperInterface.theorem4_1_true_report_formula
```

Recorded source-to-Spec screening Verdict: not recorded

Reason: not recorded

Reviewer check: ☐ Matches source input

If left unchecked, explain the discrepancy or uncertainty below.

Reviewer annotation

Review row 28

Source locator: cited publication, lines 770-773

Verbatim source input

Theorem 4.1. Any mechanism that returns an allocation with minimum possible envy cannot be truthful. The same is true for any mechanism that returns an envy-free allocation whenever there exists one.

Expanded PaperInterface specification

```
∀ (M : LMMS04FairDivision.Theorem41.LMMS41Mechanism),
  EconCSLib.FairDivision.ReturnsAllocationOf M LMMS04FairDivision.Theorem41.lmms41Goods →
  EconCSLib.FairDivision.ReturnsEnvyFreeWheneverExists M LMMS04FairDivision.Theorem41.lmms41Goods →
  ¬EconCSLib.FairDivision.DirectFairDivisionMechanism.Truthful M
```

Lean proof endpoint (built separately)

LMMS04FairDivision.PaperInterface.theorem4_1_source_not_truthful_envy_free_whenever_exists

Recorded source-to-Spec screening Verdict: not recorded

Reason: not recorded

Reviewer check: ☐ Matches source input

If left unchecked, explain the discrepancy or uncertainty below.

Reviewer annotation

Review row 29

Source locator: cited publication, lines 770-773

Verbatim source input

Theorem 4.1. Any mechanism that returns an allocation with minimum possible envy cannot be truthful. The same is true for any mechanism that returns an envy-free allocation whenever there exists one.

Expanded PaperInterface specification

```
∀ (M : LMMS04FairDivision.Theorem41.LMMS41Mechanism),
  EconCSLib.FairDivision.ReturnsMinimumReportEnvy M LMMS04FairDivision.Theorem41.lmms41Goods →
    ¬EconCSLib.FairDivision.DirectFairDivisionMechanism.Truthful M
```

Lean proof endpoint (built separately)

LMMS04FairDivision.PaperInterface.theorem4_1_source_minimum_envy_not_truthful

Recorded source-to-Spec screening Verdict: not recorded

Reason: not recorded

Reviewer check: ☐ Matches source input

If left unchecked, explain the discrepancy or uncertainty below.

Reviewer annotation

Review row 30

Source locator: cited publication, lines 758-776

Verbatim source input

In the rest of the section, we present a simple truthful mechanism which allocates the goods to the players uniformly at random. We assume that the sum of the utilities of each player over all goods is one. We will show that with high probability the maximum envy of the resulting allocation is no more than $O(\alpha \ln 1/2 + [U + 0.00F \text{ control character}])$.
Theorem 4.2. Suppose that $v_{p,i} \leq \alpha \forall p \in N, j \in M$. Then for every $[U + 0.00F \text{ control character}] > 0$, and for large enough n , there exists a truthful algorithm such that with high probability the allocation $\sqrt{\text{output}}$ by the algorithm has maximum envy at most $O(\alpha \ln 1/2 + [U + 0.00F \text{ control character}])$.

Theorem 4.1. Any mechanism that returns an allocation with minimum possible envy cannot be truthful. The same is true for any mechanism that returns an envy-free allocation whenever there exists one.

Proof. The proof is based on the probabilistic method. Allocate each good independently to player p with probability $1/n$. Clearly this is a truthful mechanism. We will show

Expanded PaperInterface specification

```
∀ {Agent : Type u_3} {Item : Type u_4} [inst : Fintype Agent] [inst_1 : Fintype Item] [inst_2 : DecidableEq Agent]
  [inst_3 : DecidableEq Item] [inst_4 : Nonempty Agent] [Nonempty Item]
  [inst_6 : Fintype (EconCSLib.FairDivision.Allocation Agent Item)]
  [inst_7 : DecidableEq (EconCSLib.FairDivision.Allocation Agent Item)],
  LMS04FairDivision.ProofInterface.randomizedTruthful
  (LMS04FairDivision.Theorem42.lmms42UniformRandomMechanism Agent Item)
```

Lean proof endpoint (built separately)

LMS04FairDivision.PaperInterface.theorem4_2_uniform_random_mechanism_truthful

Recorded source-to-Spec screening Verdict: not recorded

Reason: not recorded

Reviewer check: ☐ Matches source input

If left unchecked, explain the discrepancy or uncertainty below.

Reviewer annotation

Review row 31

Source locator: cited publication, lines 764-789

Verbatim source input

Theorem 4.2. Suppose that $\forall p, i \leq \alpha \forall p \in N, j \in M$.
Then for every $[U+000F \text{ control character}] > 0$, and for large enough n , there exists
a truthful algorithm such that with high probability the allocation
✓ output by the algorithm has maximum envy at most
 $O(\alpha n^{1/2} + [U+000F \text{ control character}])$.

Theorem 4.1. Any mechanism that returns an allocation
with minimum possible envy cannot be truthful. The same is
true for any mechanism that returns an envy-free allocation
whenever there exists one.

Proof. The proof is based on the probabilistic method.
Allocate each good independently to player p with probability $1/n$. Clearly this is a truthful mechanism. We will show
that with high probability, the allocation produced satisfies
the desired bound. Fix two players p, q . Given p and q we
define a random variable Y_j indicating the contribution of
good j to the envy of player p for q . The variable Y_j is equal
to 1, if good j is allocated to player q , -1, if it is allocated to
player p , and 0 otherwise. Hence: $Y_j = 1$ with probability
 $1/n$, -1 w.p. $1/n$ and 0 w.p.
 $P(n-2)/n$. We now define
the random variable: $fpq = \sum_j Y_j$. Clearly the envy of
 p for q is $epq = \max\{0, fpq\}$. We will show that with high
✓
probability, for every p, q , $fpq \leq O(\alpha n^{1/2} + [U+000F \text{ control character}])$ and this will
complete the proof.

Expanded PaperInterface specification

```
∀ {Agent : Type u_3} {Item : Type u_4} [inst : Fintype Agent] [inst_1 : Fintype Item] [inst_2 : DecidableEq Agent]
  [inst_3 : DecidableEq Item] [inst_4 : Nonempty Agent] [Nonempty Item]
  (w : LMMS04FairDivision.Theorem42.LMMS42ItemWeights Agent Item) {alpha t : ℝ},
  0 ≤ alpha →
  0 < t →
  (∀ (p : Agent) (g : Item), 0 ≤ w p g) →
  (∀ (p : Agent), ∑ g, w p g = 1) →
  (∀ (p : Agent) (g : Item), w p g ≤ alpha) →
  1 - 2 * alpha * ↑(Fintype.card Agent) / t ^ 2 ≤
  EconCSLib.pmfProb (LMMS04FairDivision.Theorem42.lmms42UniformAssignmentLaw Agent Item) fun assign =>
  EconCSLib.FairDivision.maxReportEnvy (LMMS04FairDivision.Theorem42.lmms42AdditiveReport w)
  (LMMS04FairDivision.Theorem42.lmms42AllocationOfAssignment assign) ≤
  t
```

Lean proof endpoint (built separately)

LMMS04FairDivision.PaperInterface.theorem4_2_uniform_random_max_envy_probability_bound

Recorded source-to-Spec screening Verdict: not recorded

Reason: not recorded

Reviewer check: ☐ Matches source input

If left unchecked, explain the discrepancy or uncertainty below.

Reviewer annotation